(MARCADOR - INÍCIO DO DOCUMENTO)

(MARCADOR - CAPA)

[NOME DA INSTITUIÇÃO - Ex: UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC]
[NOME DO CAMPUS - Ex: CAMPUS Prof. SOANE NAZARÉ DE ANDRADE]

KAYQUE COSTA MOREIRA

SECRETARIA: DESENVOLVIMENTO DE UM ASSISTENTE PESSOAL INTELIGENTE COM FOCO EM AGENDA E GERENCIAMENTO DE SENHAS

[CIDADE - ESTADO] [ANO - Ex: 2025]

(MARCADOR - FIM DA CAPA)

(MARCADOR - FOLHA DE ROSTO)

KAYQUE COSTA MOREIRA

SECRETARIA: DESENVOLVIMENTO DE UM ASSISTENTE PESSOAL INTELIGENTE COM FOCO EM AGENDA E GERENCIAMENTO DE SENHAS

Trabalho de Conclusão de Curso apresentado ao [Nome do Departamento/Colegiado] da [Nome da Instituição], como requisito parcial para obtenção do título de Bacharel em [Nome do Curso - Ex: Ciência da Computação].

Área de Concentração: [Área - Ex: Ciência da Computação]

Orientador: Prof. [Titulação] Álvaro Vinícius de Souza Coêlho

[CIDADE - ESTADO] [ANO - Ex: 2025]

(MARCADOR - FIM DA FOLHA DE ROSTO)

(MARCADOR - FOLHA DE APROVAÇÃO - Exemplo)

KAYQUE COSTA MOREIRA

SECRETARIA: DESENVOLVIMENTO DE UM ASSISTENTE PESSOAL INTELIGENTE COM FOCO EM AGENDA E GERENCIAMENTO DE SENHAS

Este Trabalho de Conclusão de Curso foi julgado adequado para a obtenção do título de Bacharel em [Nome do Curso] e aprovado em sua forma final pela Banca Examinadora:

[Local], [Dia] de [Mês] de [Ano]

Prof. [Titulação] Álvaro Vinícius de Souza Coêlho [Instituição/Departamento] (Orientador)

Prof./Dr./Me. [Nome do Membro 1 da Banca] [Instituição/Departamento]

Prof./Dr./Me. [Nome do Membro 2 da Banca] [Instituição/Departamento]

(MARCADOR - FIM DA FOLHA DE APROVAÇÃO)

(MARCADOR - AGRADECIMENTOS - Exemplo)

AGRADECIMENTOS

(Espaço para o texto de agradecimentos do aluno. Exemplo: Agradeço primeiramente a Deus... Aos meus pais... Ao meu orientador, Prof. Álvaro Vinícius de Souza Coêlho, pela paciência e direcionamento... Aos colegas... À instituição...)

(MARCADOR - FIM DOS AGRADECIMENTOS)

(MARCADOR - RESUMO)

RESUMO

(Espaço para o Resumo do TCC. Será necessário redigir um resumo conciso após a finalização do conteúdo principal, abordando contextualização, problema, objetivos, metodologia, principais resultados e conclusões. Exemplo baseado no TCC exemplo: Este trabalho descreve o desenvolvimento da "secretarIA", uma aplicação web que visa integrar gerenciamento de agenda, cofre de senhas e interação conversacional via IA

para otimizar a produtividade pessoal. Dada a fragmentação de ferramentas digitais e os avanços em LLMs, a secretarIA foi projetada para centralizar tarefas e oferecer uma interface mais natural. O desenvolvimento seguiu uma abordagem iterativa, utilizando Next.js, React, Node.js, MongoDB, LangChain e a API da OpenAI. A arquitetura foi baseada em camadas, visando manutenibilidade e escalabilidade. A avaliação do protótipo incluiu análise funcional, avaliação heurística de usabilidade e análise da interação com a IA. Os resultados indicam sucesso na implementação das funcionalidades básicas e potencial na interação via IA, apesar das limitações identificadas na compreensão de linguagem natural complexa. O trabalho contribui com um estudo de caso sobre a integração de LLMs em IPAs especializados e discute desafios de UX, segurança e privacidade.)

Palavras-chave: Assistente Pessoal Inteligente. Modelo de Linguagem Grande. LangChain. Gerenciamento de Agenda. Gerenciador de Senhas. Next.js. Experiência do Usuário.

(MARCADOR - FIM DO RESUMO)

(MARCADOR - ABSTRACT)

ABSTRACT

(Espaço para a versão em inglês do Resumo. Exemplo: This work describes the development of "secretarIA", a web application aimed at integrating calendar management, password vault, and conversational AI interaction to optimize personal productivity. Given the fragmentation of digital tools and advances in LLMs, secretarIA was designed to centralize tasks and offer a more natural interface. Development followed an iterative approach using Next.js, React, Node.js, MongoDB, LangChain, and the OpenAI API. The architecture was layer-based, focusing on maintainability and scalability. Prototype evaluation included functional analysis, heuristic usability assessment, and AI interaction analysis. Results indicate successful implementation of core features and potential in AI interaction, despite identified limitations in understanding complex natural language. The work contributes a case study on integrating LLMs into specialized IPAs and discusses UX, security, and privacy challenges.)

Keywords: Intelligent Personal Assistant. Large Language Model. LangChain. Calendar Management. Password Manager. Next.js. User Experience.

(MARCADOR - FIM DO ABSTRACT)

(MARCADOR - LISTA DE ABREVIATURAS E SIGLAS - Exemplo)

LISTA DE ABREVIATURAS E SIGLAS

AES Advanced Encryption Standard API Application Programming Interface ASR Automatic Speech Recognition CRUD Create, Read, Update, Delete CSR Client-Side Rendering CSS Cascading Style Sheets CUI Chat User Interface E2EE End-to-End Encryption GPT Generative Pre-trained Transformer HTML HyperText Markup Language HTTP Hypertext Transfer Protocol HTTPS Hypertext Transfer Protocol Secure IA Inteligência Artificial IPA Intelligent Personal Assistant (Assistente Pessoal Inteligente) IR Information Retrieval IV Initialization Vector JWT JSON Web Token LLM Large Language Model (Modelo de Linguagem Grande) LGPD Lei Geral de Proteção de Dados Pessoais ML Machine Learning NLG Natural Language Generation NLU Natural Language Understanding NoSQL Not Only SQL OAuth Open Authorization ODM Object-Document Mapper OWASP Open Web Application Security Project PLN Processamento de Linguagem Natural REST Representational State Transfer RF Requisito Funcional RNF Requisito Não Funcional SSR Server-Side Rendering SQL Structured Query Language TTS Text-to-Speech UI User Interface URI Uniform Resource Identifier URL Uniform Resource Locator UX User Experience VUI Voice User Interface XAI Explainable Artificial Intelligence XSS Cross-Site Scripting

(MARCADOR - FIM DA LISTA DE ABREVIATURAS E SIGLAS)

(MARCADOR - LISTA DE FIGURAS)

LISTA DE FIGURAS

(Esta lista será preenchida automaticamente pelo LaTeX/Word ou manualmente após a inserção de todas as figuras. Exemplo de como ficará após a inserção:)

Figura 1: Tela de Login da Aplicação secretarIA..... [Página] Figura 2: Tela de Cadastro da Aplicação secretarIA..... [Página] Figura 3: Visualização Mensal do Módulo Calendário..... [Página] Figura 4: Modal de Adição/Edição de Evento..... [Página] Figura 5: Visualização Anual do Módulo Calendário..... [Página] Figura 6: Visualização em Lista (Agenda) do Módulo Calendário..... [Página] Figura 7: Interface do Módulo Chat..... [Página] Figura 8: Exemplo de Interação de Adição de Evento via Chat..... [Página] Figura 9: Calendário Atualizado Após Interação via Chat..... [Página] Figura 10: Exemplo de Edição de Evento..... [Página] Figura 11: Calendário Após Reagendamento..... [Página] Figura 12: Exemplo de Exclusão de Evento..... [Página] Figura 13: Calendário Após Exclusão..... [Página] Figura 14: Exemplo de E-mail de Notificação (se aplicável)..... [Página] Figura 15: Interface do Módulo Gerenciador de Senhas..... [Página] Figura 16: Modal de Adição/Edição de Credencial..... [Página] Figura 17: Lista de Senhas Após Cadastro..... [Página] Figura 18: Exemplo de Estrutura de Dados no MongoDB (Senhas)..... [Página] Figura 19: Exemplo de Estrutura de Dados no MongoDB (Eventos)..... [Página] Figura 20: Exemplo de Estrutura de Dados no MongoDB (Chats - se aplicável)..... [Página] Figura 21: Exemplo de Estrutura de Dados no MongoDB (Usuários)..... [Página] Figura 22: Diagrama de Arquitetura Detalhado da secretarIA..... [Página]

(MARCADOR - FIM DA LISTA DE FIGURAS)

(MARCADOR - SUMÁRIO)

SUMÁRIO

(Este sumário será preenchido automaticamente pelo LaTeX/Word ou manualmente após a finalização. Exemplo de estrutura:)

1
INTRODUÇÃO.....
[Página] 1.1 Contextualização: A Ascensão da Inteligência Artificial Conversacional no Cotidiano..... [Página] 1.2 O Problema: Gerenciamento de Agendas na Era Digital e as Limitações Atuais..... [Página] 1.3 Objetivos do Trabalho..... [Página] 1.4 Justificativa e Relevância..... [Página] 1.5 Estrutura do Trabalho..... [Página]

2 FUNDAMENTAÇÃO TEÓRICA EXPANDIDA.....
[Página] 2.1 Assistentes Pessoais Inteligentes (IPAs)..... [Página] 2.1.1 Histórico e Evolução..... [Página] 2.1.2 Definições e Conceitos Chave..... [Página] 2.1.3 Arquiteturas Comuns..... [Página] 2.1.4 Tecnologias Fundamentais..... [Página] 2.1.5 Tipos de IPAs..... [Página] 2.1.6 Aplicações e Domínios..... [Página] 2.2 Modelos de Linguagem Grandes (LLMs)..... [Página] 2.2.1 O que são LLMs?..... [Página] 2.2.2 Arquitetura Transformer..... [Página] 2.2.3 Capacidades e Limitações..... [Página] 2.2.4 Prompt Engineering..... [Página] 2.3 Framework LangChain..... [Página] 2.3.1 Motivação e Objetivos..... [Página] 2.3.2 Componentes Principais (Models, Prompts, Chains, Agents, Memory, Indexes)..... [Página] 2.3.3 Aplicações e Casos de Uso..... [Página] 2.4 Desenvolvimento Web Moderno: Next.js e Ecossistema..... [Página] 2.4.1 React e Componentização..... [Página] 2.4.2 Node.js e JavaScript no Backend..... [Página] 2.4.3 Next.js: Vantagens e Funcionalidades..... [Página] 2.4.4 Tailwind CSS: Abordagem Utility-First..... [Página] 2.5 Bancos de Dados NoSQL: MongoDB..... [Página] 2.5.1 Conceitos NoSQL e Modelo de Documentos..... [Página] 2.5.2 Vantagens e Desvantagens..... [Página] 2.5.3 Mongoose ODM..... [Página] 2.6 Segurança em Aplicações Web..... [Página] 2.6.1

Autenticação e Autorização (OAuth, JWT, NextAuth.js)..... [Página] 2.6.2 Criptografia (Hashing, Simétrica - AES)..... [Página] 2.6.3 Principais Vulnerabilidades (OWASP Top 10) e Mitigações..... [Página] 2.7 Privacidade e Ética em IA..... [Página] 2.7.1 Preocupações com Privacidade em IPAs e LLMs..... [Página] 2.7.2 Vieses em Modelos de IA..... [Página] 2.7.3 Transparência e Explicabilidade (XAI)..... [Página] 2.7.4 Regulamentações (LGPD)..... [Página] 2.8 Experiência do Usuário (UX) em Interfaces Conversacionais..... [Página] 2.8.1 Princípios de Design de CUI/VUI..... [Página] 2.8.2 Desafios de Usabilidade em Chatbots e IPAs..... [Página] 2.8.3 Avaliação de Usabilidade (Heurísticas de Nielsen)..... [Página]

3 METODOLOGIA EXPANDIDA..... [Página] 3.1 Abordagem Metodológica..... [Página] 3.2 Desenho Metodológico Detalhado..... [Página] 3.3 Ferramentas e Tecnologias: Justificativas Detalhadas..... [Página] 3.4 Coleta e Análise de Dados (Avaliação do Protótipo)..... [Página] 3.5 Considerações Éticas..... [Página]

4 DESENVOLVIMENTO DA APLICAÇÃO SECRETARIA..... [Página] 4.1 Análise de Requisitos Detalhada..... [Página] 4.1.1 Requisitos Funcionais (RF)..... [Página] 4.1.2 Requisitos Não Funcionais (RNF)..... [Página] 4.2 Arquitetura do Sistema Detalhada..... [Página] 4.3 Implementação das Funcionalidades Chave..... [Página] 4.4 Implementação de Segurança e Privacidade..... [Página] 4.5 Desafios Enfrentados e Soluções Adotadas..... [Página]

5 RESULTADOS E DISCUSSÃO..... [Página] 5.1 Apresentação Detalhada da Aplicação Final..... [Página] 5.2 Análise Funcional Detalhada..... [Página] 5.3 Análise da Interação Conversacional (IA) Aprofundada..... [Página] 5.4 Avaliação Heurística de Usabilidade Detalhada..... [Página] 5.5 Discussão Aprofundada sobre Segurança e Privacidade..... [Página]

6
CONCLUSÃO..... [Página] 6.1 Síntese Abrangente do Trabalho..... [Página] 6.2 Resposta aos Objetivos Específicos..... [Página] 6.3 Principais Contribuições..... [Página] 6.4 Implicações Práticas e Teóricas..... [Página] 6.5 Limitações e Desafios Remanescentes..... [Página] 6.6 Recomendações Detalhadas para Trabalhos Futuros..... [Página] 6.7 Considerações Finais e Visão de Futuro..... [Página]

REFERÊNCIAS BIBLIOGRÁFICAS..... [Página]

(APÊNDICES - se houver)..... [Página]

(MARCADOR - FIM DO SUMÁRIO)

(MARCADOR - INÍCIO DO CONTEÚDO PRINCIPAL)

Capítulo 1: Introdução Aprofundada (Rascunho Inicial)

1.1 Contextualização: A Ascensão da Inteligência Artificial Conversacional no Cotidiano

Vivemos em uma era marcada pela crescente integração da inteligência artificial (IA) em nossas vidas diárias. De sistemas de recomendação que moldam nosso consumo de mídia a algoritmos que otimizam rotas de trânsito, a IA deixou de ser um conceito futurista para se tornar uma ferramenta onipresente, embora muitas vezes invisível.

Dentro desse vasto espectro, uma das manifestações mais tangíveis e interativas da IA é a inteligência artificial conversacional, materializada principalmente nos Assistentes Pessoais Inteligentes (IPAs), como a Siri da Apple, o Google Assistente e a Alexa da Amazon (Silva et al., 2020).

Esses assistentes, impulsionados por avanços notáveis no Processamento de Linguagem Natural (PLN) e nos Modelos de Linguagem Grandes (LLMs), transformaram a maneira como interagimos com a tecnologia. A capacidade de comandar dispositivos, buscar informações, gerenciar tarefas e até mesmo manter diálogos fluidos usando apenas a voz ou texto natural representa uma mudança paradigmática na interação humano-computador (Han & Yang, 2018; Kundu, 2023). A promessa é a de uma tecnologia mais intuitiva, acessível e integrada ao fluxo natural das atividades humanas.

No entanto, essa rápida proliferação não ocorre sem seus desafios. Questões relacionadas à privacidade e segurança dos dados tornam-se cada vez mais prementes, à medida que esses sistemas coletam e processam volumes crescentes de informações pessoais (Bolton et al., 2021; Mols et al., 2021). A qualidade da experiência do usuário (UX) ainda apresenta inconsistências, com limitações na compreensão contextual e na capacidade de lidar com tarefas mais complexas ou nuances da comunicação humana (Budiu, 2018; Lopatovska et al., 2020). Além disso, preocupações éticas sobre vieses algorítmicos, transparência e o potencial impacto social da automação impulsionada por IA conversacional exigem uma reflexão crítica contínua (Piñeiro-Martín et al., 2023; Rillig et al., 2024).

É neste cenário complexo e dinâmico – de grande potencial tecnológico, mas também de desafios significativos – que se insere o presente trabalho.

1.2 O Problema: Gerenciamento de Agendas na Era Digital e as Limitações Atuais

A gestão eficaz do tempo e das tarefas é um desafio perene na vida moderna, tanto no âmbito profissional quanto pessoal. A proliferação de compromissos, reuniões, prazos e eventos sociais exige ferramentas que auxiliem na organização e no planejamento. Embora existam inúmeras aplicações de calendário e gerenciamento de tarefas, muitas delas ainda dependem de interfaces gráficas tradicionais, exigindo interação manual para adicionar, consultar ou modificar eventos.

Os assistentes pessoais inteligentes existentes oferecem algumas funcionalidades de gerenciamento de agenda, como adicionar eventos por comando de voz ou ler os compromissos do dia. Contudo, frequentemente carecem de uma compreensão mais profunda do contexto do usuário, da capacidade de realizar consultas complexas sobre a

disponibilidade ou de integrar informações de forma proativa. A interação pode ser fragmentada, exigindo comandos precisos e, por vezes, falhando em interpretar solicitações mais naturais ou ambíguas. Além disso, a integração com outras ferramentas de produtividade ou a personalização avançada podem ser limitadas.

A lacuna identificada reside na necessidade de um assistente mais especializado e contextualmente ciente para o gerenciamento de agendas, que vá além das funcionalidades básicas oferecidas pelos IPAs generalistas. Um assistente que não apenas registre compromissos, mas que possa responder a perguntas complexas sobre a agenda, entender a disponibilidade do usuário em diferentes cenários e interagir de forma mais natural e eficiente, funcionando como uma verdadeira "secretária" digital pessoal.

1.3 Objetivos do Trabalho

Diante do contexto e do problema apresentados, este Trabalho de Conclusão de Curso (TCC) tem como objetivo geral projetar, desenvolver e analisar uma aplicação web denominada "secretarIA", que atua como uma assistente pessoal inteligente especializada no gerenciamento de agendas e senhas, utilizando tecnologias modernas de desenvolvimento web e inteligência artificial conversacional.

Para alcançar este objetivo geral, foram definidos os seguintes objetivos específicos:

1. **Pesquisar e Analisar:** Investigar o estado da arte em assistentes pessoais inteligentes, modelos de linguagem grandes (LLMs), frameworks de desenvolvimento de IA conversacional (como LangChain), tecnologias de desenvolvimento web (Next.js, Node.js, Express) e princípios de design de interação e segurança aplicáveis ao contexto do projeto.
2. **Projetar a Arquitetura:** Definir a arquitetura de software da aplicação secretarIA, detalhando os componentes de frontend, backend, a integração com a API de um LLM (OpenAI GPT-3.5-turbo ou similar) e o banco de dados para armazenamento de dados do usuário (eventos de calendário e senhas criptografadas).
3. **Desenvolver a Aplicação:** Implementar as funcionalidades principais da secretarIA, incluindo:
    - Interface de calendário visualmente intuitiva.
    - Interface de chat para interação conversacional com a IA para gerenciamento da agenda.
    - Funcionalidade de gerenciamento seguro de senhas.
    - Autenticação e autorização de usuários.
4. **Integrar a Inteligência Artificial:** Implementar a lógica de IA conversacional utilizando um LLM e o framework LangChain para permitir que os usuários

consultem e gerenciem suas agendas através de linguagem natural na interface de chat.
5. **Avaliar e Analisar:** Realizar uma análise crítica da aplicação desenvolvida, discutindo suas funcionalidades, limitações, desafios encontrados durante o desenvolvimento e potenciais melhorias, à luz da fundamentação teórica pesquisada, com foco particular nos aspectos de usabilidade, segurança e as capacidades/limitações da IA implementada.

1.4 Justificativa e Relevância

A relevância deste trabalho reside em múltiplos aspectos:

- **Tecnológica:** Explora a aplicação prática de tecnologias de ponta, como LLMs e frameworks como LangChain, em um domínio específico (gerenciamento de agendas), contribuindo para a compreensão de suas potencialidades e desafios em cenários reais.
- **Funcional:** Propõe uma solução para um problema cotidiano – a gestão de agendas – buscando oferecer uma interação mais natural e eficiente através da IA conversacional, potencialmente superando limitações de ferramentas existentes.
- **Acadêmica:** Sistematiza o processo de desenvolvimento de uma aplicação web complexa que integra IA, servindo como um estudo de caso e fornecendo insights sobre a arquitetura, as ferramentas e as metodologias empregadas. Além disso, aprofunda a discussão sobre temas cruciais como UX, privacidade e ética no contexto de IPAs.
- **Pessoal:** Representa a consolidação dos conhecimentos adquiridos pelo autor ao longo do curso, aplicando-os em um projeto desafiador e com potencial de utilidade prática.

A crescente demanda por soluções de produtividade personalizadas e a rápida evolução da IA conversacional tornam o desenvolvimento da secretarIA um projeto oportuno e relevante, capaz de gerar conhecimento tanto técnico quanto conceitual.

1.5 Estrutura do Trabalho

Este documento está organizado da seguinte forma:

- **Capítulo 1 (Introdução):** Apresenta o contexto geral da IA conversacional, define o problema de pesquisa, estabelece os objetivos, justifica a relevância do trabalho e descreve a estrutura do documento.

- **Capítulo 2 (Fundamentação Teórica):** Revisa a literatura relevante, abordando conceitos de IPAs, PLN, LLMs, frameworks (LangChain), arquitetura de software, padrões de design, UX, privacidade, segurança e ética em IA.
- **Capítulo 3 (Metodologia):** Detalha a abordagem metodológica adotada para o desenvolvimento do projeto, incluindo as fases de pesquisa, design, implementação e análise.
- **Capítulo 4 (Desenvolvimento):** Descreve em detalhes o processo de desenvolvimento da aplicação secretarIA, incluindo a arquitetura implementada, as tecnologias utilizadas, as funcionalidades desenvolvidas e os desafios encontrados.
- **Capítulo 5 (Resultados e Discussão):** Apresenta a aplicação finalizada e discute os resultados alcançados em relação aos objetivos propostos, analisando as funcionalidades, limitações e contribuições à luz da fundamentação teórica.
- **Capítulo 6 (Conclusão):** Sumariza as principais conclusões do trabalho, recapitula as contribuições, aponta as limitações e sugere direções para trabalhos futuros.

Seguem-se as Referências Bibliográficas e possíveis Apêndices com informações complementares.

Capítulo 2: Fundamentação Teórica Expandida (Rascunho v3)

Este capítulo estabelece as bases conceituais e teóricas que sustentam o desenvolvimento da aplicação secretarIA. Abordaremos os principais elementos tecnológicos e metodológicos envolvidos, desde a natureza dos Assistentes Pessoais Inteligentes (IPAs) e dos Modelos de Linguagem Grandes (LLMs) que os impulsionam, até as considerações cruciais sobre experiência do usuário (UX) em interfaces conversacionais, segurança, privacidade e as arquiteturas de software que viabilizam tais sistemas. A compreensão aprofundada desses tópicos é essencial para contextualizar as decisões de projeto, os desafios enfrentados e os resultados obtidos neste trabalho.

2.1 Assistentes Pessoais Inteligentes (IPAs)

Os Assistentes Pessoais Inteligentes (IPAs), também conhecidos como assistentes virtuais ou agentes conversacionais, representam uma evolução significativa na interação humano-computador. Eles são sistemas de software projetados para auxiliar

os usuários na realização de tarefas ou na obtenção de informações através de interfaces de linguagem natural, predominantemente voz ou texto (Silva et al., 2020).

2.1.1 Histórico e Evolução

A ideia de máquinas que compreendem e respondem à linguagem humana remonta aos primórdios da inteligência artificial. Projetos iniciais como ELIZA (Weizenbaum, 1966) simulavam conversas, embora de forma rudimentar. O desenvolvimento de tecnologias de reconhecimento de fala e processamento de linguagem natural (PLN) nas décadas seguintes pavimentou o caminho para assistentes mais sofisticados. A introdução do Siri pela Apple em 2011 marcou um ponto de inflexão, popularizando os IPAs em dispositivos móveis. Logo depois, surgiram concorrentes como Google Assistant, Amazon Alexa e Microsoft Cortana, cada um expandindo as capacidades e a onipresença desses assistentes em smartphones, smart speakers e outros dispositivos (Hoy, 2018).

Inicialmente focados em tarefas simples como definir alarmes, fazer buscas na web ou tocar música, os IPAs evoluíram para integrar-se a ecossistemas de dispositivos domésticos inteligentes (Internet das Coisas - IoT), realizar transações, gerenciar calendários e até mesmo oferecer suporte em áreas especializadas como saúde e educação (Silva et al., 2020; Bérubé et al., 2021).

2.1.2 Definições e Conceitos Chave

Um IPA pode ser definido como um sistema que utiliza inteligência artificial, particularmente PLN e aprendizado de máquina, para compreender as intenções do usuário expressas em linguagem natural e executar ações correspondentes ou fornecer informações relevantes (Silva et al., 2020). Os componentes chave geralmente incluem:

- **Interface de Usuário:** Tipicamente baseada em voz (Voice User Interface - VUI) ou texto (Chat User Interface - CUI).
- **Módulo de Compreensão de Linguagem Natural (NLU):** Responsável por analisar a entrada do usuário, identificar intenções e extrair entidades relevantes (datas, locais, nomes, etc.).
- **Gerenciador de Diálogo:** Mantém o estado da conversa, decide a próxima ação do sistema e formula a resposta.
- **Módulo de Geração de Linguagem Natural (NLG):** Converte a resposta do sistema em linguagem natural (texto ou fala).
- **Base de Conhecimento/Integração com Serviços:** Acessa informações internas ou se conecta a APIs externas para buscar dados ou executar tarefas.

2.1.3 Arquiteturas Comuns

Silva et al. (2020), em sua revisão sistemática, identificam diversas abordagens arquiteturais para IPAs. Muitas arquiteturas modernas são baseadas em nuvem, onde o processamento pesado de PLN e IA ocorre em servidores remotos, permitindo que os dispositivos clientes sejam mais leves. Arquiteturas comuns envolvem pipelines de processamento que incluem reconhecimento de fala (ASR - Automatic Speech Recognition), NLU, gerenciamento de diálogo, NLG e síntese de fala (TTS - Text-to-Speech). A ascensão dos LLMs está influenciando essas arquiteturas, potencialmente simplificando alguns desses módulos ao concentrar mais inteligência no próprio modelo de linguagem.

2.1.4 Tecnologias Fundamentais

O funcionamento dos IPAs depende de um conjunto de tecnologias de IA:

- **Processamento de Linguagem Natural (PLN):** O campo guarda-chuva que engloba técnicas para permitir que computadores processem e compreendam a linguagem humana. Inclui NLU e NLG.
- **Reconhecimento de Fala (ASR):** Converte áudio de voz em texto.
- **Síntese de Fala (TTS):** Converte texto em áudio de voz.
- **Aprendizado de Máquina (Machine Learning - ML):** Utilizado para treinar os modelos de NLU, ASR, TTS e, cada vez mais, os próprios gerenciadores de diálogo e LLMs subjacentes.
- **Recuperação de Informação (Information Retrieval - IR):** Técnicas para buscar informações relevantes em grandes bases de dados ou na web.

2.1.5 Tipos de IPAs

Os IPAs podem ser classificados de diversas formas:

- **Generalistas vs. Especializados:** IPAs como Alexa e Google Assistant são generalistas, projetados para lidar com uma ampla gama de tarefas. Outros são especializados em domínios específicos, como saúde (Bérubé et al., 2021), finanças ou, como no caso da secretarIA, gerenciamento de agenda e senhas.
- **Reativos vs. Proativos:** A maioria dos IPAs atuais é reativa, respondendo a comandos explícitos do usuário. Há um interesse crescente em IPAs proativos, que antecipam as necessidades do usuário e oferecem sugestões ou realizam ações sem serem solicitados.
- **Baseados em Regras vs. Baseados em Aprendizado:** Abordagens mais antigas dependiam fortemente de regras manuais para diálogo e NLU. Abordagens modernas são predominantemente baseadas em aprendizado de máquina.

2.1.6 Aplicações e Domínios

O uso de IPAs se expandiu para inúmeros domínios:

- **Produtividade Pessoal:** Gerenciamento de calendários, listas de tarefas, e-mails, lembretes.
- **Casa Inteligente:** Controle de luzes, termostatos, fechaduras, eletrodomésticos.
- **Entretenimento:** Tocar música, podcasts, audiolivros, controlar TVs.
- **Informação:** Notícias, previsão do tempo, resultados esportivos, buscas na web.
- **Comércio:** Compras online, pedidos de comida.
- **Saúde:** Monitoramento de sintomas, lembretes de medicação, acesso a informações de saúde (Ermolina et al., 2021).
- **Educação:** Auxílio no aprendizado de idiomas, resposta a perguntas de estudantes (Gubareva & Lopes, 2020).
- **Acessibilidade:** Ferramenta crucial para pessoas com deficiências visuais ou motoras (Noel, 2020).

O projeto secretarIA se insere principalmente no domínio da produtividade pessoal, com foco específico em agenda e senhas.

2.2 Modelos de Linguagem Grandes (LLMs)

Os Modelos de Linguagem Grandes (Large Language Models - LLMs) representam um avanço disruptivo no campo da IA e do PLN, sendo a tecnologia central por trás das capacidades conversacionais de muitos IPAs modernos, incluindo a secretarIA.

2.2.1 O que são LLMs?

LLMs são modelos de aprendizado profundo, tipicamente baseados na arquitetura Transformer (Vaswani et al., 2017), treinados em volumes massivos de dados textuais (trilhões de palavras ou mais) provenientes da internet e de outras fontes. Seu objetivo principal é aprender padrões estatísticos complexos da linguagem para poder prever a próxima palavra (ou token) em uma sequência, dada uma sequência anterior (Zhao et al., 2023). Essa capacidade aparentemente simples, quando escalada para modelos com bilhões ou trilhões de parâmetros e treinada em dados vastos, resulta em habilidades emergentes notáveis, como:

- **Geração de Texto Coerente:** Produzir textos longos e contextualmente relevantes sobre uma ampla gama de tópicos.
- **Compreensão de Linguagem Natural:** Entender perguntas, instruções e nuances da linguagem humana.
- **Tradução:** Traduzir textos entre diferentes idiomas.

- **Sumarização:** Resumir textos longos em versões mais curtas.
- **Resposta a Perguntas:** Responder a perguntas factuais ou baseadas em um contexto fornecido.
- **Raciocínio (Limitado):** Realizar formas simples de raciocínio lógico ou matemático.
- **Geração de Código:** Escrever código em várias linguagens de programação.

Exemplos proeminentes de LLMs incluem a família GPT (Generative Pre-trained Transformer) da OpenAI (como GPT-3.5 e GPT-4), LaMDA e PaLM do Google, LLaMA da Meta, e modelos open-source como BLOOM e Falcon.

2.2.2 Arquitetura Transformer

A arquitetura Transformer, introduzida por Vaswani et al. (2017), foi fundamental para o sucesso dos LLMs. Ela se baseia principalmente no mecanismo de **atenção** (especificamente, self-attention), que permite ao modelo ponderar a importância de diferentes partes da sequência de entrada ao processar cada parte. Isso permite capturar dependências de longo alcance no texto de forma mais eficaz do que arquiteturas anteriores como RNNs ou LSTMs.

Os principais componentes do Transformer são:

- **Embeddings:** Converte tokens de entrada (palavras ou subpalavras) em vetores numéricos.
- **Positional Encoding:** Adiciona informações sobre a posição dos tokens na sequência, já que a arquitetura em si não processa sequencialmente.
- **Multi-Head Self-Attention:** Permite que o modelo preste atenção a diferentes partes da sequência de entrada simultaneamente e de diferentes maneiras (múltiplas "cabeças" de atenção).
- **Add & Norm (Residual Connections and Layer Normalization):** Ajuda a estabilizar o treinamento de redes profundas.
- **Feed-Forward Networks:** Redes neurais totalmente conectadas aplicadas independentemente a cada posição.

LLMs modernos geralmente consistem em empilhar dezenas desses blocos Transformer.

2.2.3 Capacidades e Limitações

Apesar de suas capacidades impressionantes, os LLMs possuem limitações importantes (Zhao et al., 2023; Kundu, 2023):

- **Alucinações:** Podem gerar informações factualmente incorretas ou sem sentido, mas apresentá-las de forma confiante.

- **Falta de Conhecimento Atualizado:** Seu conhecimento é limitado aos dados com os quais foram treinados, que podem estar desatualizados.
- **Vieses:** Podem herdar e amplificar vieses presentes nos dados de treinamento (sociais, culturais, etc.).
- **Raciocínio Complexo:** Lutam com tarefas que exigem raciocínio multi-passo complexo, planejamento ou conhecimento de mundo profundo.
- **Segurança e Uso Malicioso:** Podem ser usados para gerar desinformação, spam ou conteúdo prejudicial.
- **Custo Computacional:** Treinar e executar LLMs de grande escala exige recursos computacionais significativos.
- **Compreensão Contextual Limitada:** Embora melhores que modelos anteriores, ainda podem perder o contexto em conversas longas ou complexas.

2.2.4 Prompt Engineering

Dado que os LLMs são altamente sensíveis à forma como as instruções são fornecidas, a **engenharia de prompts** (Prompt Engineering) tornou-se uma disciplina crucial. Trata-se da arte e ciência de formular entradas (prompts) eficazes para guiar o LLM a produzir a saída desejada. Isso pode envolver:

- **Instruções Claras:** Definir a tarefa e o formato de saída esperado.
- **Fornecimento de Contexto:** Incluir informações relevantes que o modelo precisa para realizar a tarefa.
- **Exemplos (Few-Shot Learning):** Dar ao modelo alguns exemplos de entrada e saída desejada.
- **Definição de Persona:** Instruir o modelo a agir como um determinado personagem ou especialista.
- **Técnicas Avançadas:** Como Chain-of-Thought (CoT), que instrui o modelo a explicar seu raciocínio passo a passo antes de dar a resposta final.

No contexto da secretarIA, a engenharia de prompts é essencial para instruir o LLM sobre seu papel, fornecer o contexto da agenda do usuário e formatar a consulta para obter respostas úteis e ações corretas.

2.3 Framework LangChain

Construir aplicações complexas sobre LLMs pode exigir mais do que apenas chamadas diretas à API do modelo. É preciso gerenciar prompts, conectar LLMs a outras fontes de dados, manter o estado da conversa e orquestrar fluxos de trabalho mais elaborados. O LangChain surge como um framework popular para facilitar esse processo.

2.3.1 Motivação e Objetivos

O LangChain visa fornecer um conjunto de ferramentas e abstrações para simplificar o desenvolvimento de aplicações baseadas em LLMs. Seus principais objetivos são:

- **Modularidade:** Oferecer componentes reutilizáveis para diferentes partes do pipeline de uma aplicação LLM.
- **Composibilidade:** Permitir que esses componentes sejam facilmente combinados para criar cadeias (chains) e agentes complexos.
- **Integração:** Facilitar a conexão com diversos modelos de LLM, fontes de dados e ferramentas externas.
- **Gerenciamento de Estado:** Fornecer mecanismos para lidar com a memória em conversas.

2.3.2 Componentes Principais

O LangChain é organizado em torno de vários módulos chave:

- **Models:** Interface unificada para interagir com diferentes tipos de modelos (LLMs, Chat Models, Text Embedding Models).
- **Prompts:** Ferramentas para gerenciar e otimizar prompts, incluindo templates de prompt, seletores de exemplo e formatação de saída.
- **Chains:** Sequências de chamadas (a LLMs ou outras ferramentas) que permitem construir aplicações mais complexas. Exemplos incluem `LLMChain` (a mais simples, combina prompt e modelo), cadeias sequenciais, e cadeias para sumarização ou resposta a perguntas sobre documentos.
- **Agents:** Sistemas onde um LLM atua como um "cérebro" que toma decisões sobre quais ferramentas usar (ex: busca na web, calculadora, acesso a APIs) para responder a uma consulta, executando um ciclo de pensamento-ação-observação.
- **Memory:** Mecanismos para persistir o estado entre chamadas em uma cadeia ou agente, permitindo que as aplicações tenham memória conversacional (ex: `ConversationBufferMemory`).
- **Indexes (ou Retrievers):** Formas de estruturar documentos e dados para que os LLMs possam interagir com eles de forma eficiente, como na técnica de Retrieval-Augmented Generation (RAG), onde informações relevantes são recuperadas de uma base de dados vetorial e injetadas no prompt do LLM.

2.3.3 Aplicações e Casos de Uso

O LangChain é utilizado para construir uma variedade de aplicações, incluindo:

- **Chatbots Conversacionais:** Com memória e capacidade de responder a perguntas complexas.
- **Sumarização de Documentos:** Processar textos longos e gerar resumos.
- **Question Answering sobre Dados:** Responder perguntas com base em documentos específicos ou bases de dados.
- **Extração de Informação:** Identificar e extrair dados estruturados de texto não estruturado.
- **Agentes Autônomos:** Que podem interagir com o ambiente digital para realizar tarefas.

No projeto secretarIA, o LangChain foi utilizado principalmente para gerenciar os prompts enviados ao LLM da OpenAI, incorporar o contexto da agenda do usuário nesses prompts e, potencialmente, gerenciar a memória da conversa na interface de chat (utilizando `PromptTemplate`, `ChatOpenAI`, `LLMChain` e talvez módulos de `Memory`).

2.4 Desenvolvimento Web Moderno: Next.js e Ecossistema

A escolha da stack tecnológica para o frontend e backend é crucial para a eficiência do desenvolvimento, a qualidade da aplicação final e sua manutenibilidade. A secretarIA foi desenvolvida utilizando Next.js, um framework popular construído sobre React e Node.js.

2.4.1 React e Componentização

React é uma biblioteca JavaScript para construir interfaces de usuário (UI), conhecida por seu modelo baseado em componentes e seu Virtual DOM. A componentização permite dividir a UI em peças reutilizáveis e independentes, facilitando o desenvolvimento, teste e manutenção. O Virtual DOM otimiza as atualizações da UI, melhorando o desempenho.

2.4.2 Node.js e JavaScript no Backend

Node.js é um ambiente de execução JavaScript do lado do servidor, permitindo que desenvolvedores usem a mesma linguagem tanto no frontend quanto no backend. Sua natureza assíncrona e orientada a eventos o torna adequado para aplicações web que

lidam com muitas operações de I/O (como chamadas a bancos de dados e APIs externas).

2.4.3 Next.js: Vantagens e Funcionalidades

Next.js é um framework React que adiciona funcionalidades importantes para construir aplicações web prontas para produção:

- **Renderização Híbrida:** Suporta diferentes estratégias de renderização por página: Server-Side Rendering (SSR), Static Site Generation (SSG), Incremental Static Regeneration (ISR) e Client-Side Rendering (CSR). Isso permite otimizar o desempenho e o SEO.
- **Roteamento Baseado em Arquivos:** Simplifica a criação de rotas na aplicação.
- **API Routes:** Permite criar endpoints de API diretamente dentro do projeto Next.js, facilitando o desenvolvimento full-stack.
- **Otimização Automática:** Inclui otimização de imagens, code splitting automático e outras melhorias de desempenho.
- **Ecossistema:** Integra-se bem com outras ferramentas populares do ecossistema React e Node.js.

Para a secretarIA, o Next.js ofereceu uma solução integrada para construir tanto a interface do usuário quanto a API backend que se comunica com o banco de dados e a IA.

2.4.4 Tailwind CSS: Abordagem Utility-First

Tailwind CSS é um framework CSS "utility-first" que fornece classes de baixo nível (utilitários) que podem ser compostas diretamente no HTML para construir designs customizados sem escrever CSS tradicional. Isso acelera o desenvolvimento da UI e promove a consistência visual.

2.5 Bancos de Dados NoSQL: MongoDB

Para armazenar os dados da aplicação (usuários, eventos, senhas), foi escolhido o MongoDB, um banco de dados NoSQL orientado a documentos.

2.5.1 Conceitos NoSQL e Modelo de Documentos

Bancos de dados NoSQL (Not Only SQL) oferecem alternativas aos bancos de dados relacionais tradicionais (SQL). O MongoDB armazena dados em documentos flexíveis do tipo BSON (Binary JSON), agrupados em coleções. Esse modelo permite esquemas dinâmicos, onde documentos na mesma coleção não precisam ter a mesma estrutura.

2.5.2 Vantagens e Desvantagens

- **Vantagens:** Flexibilidade de esquema, escalabilidade horizontal mais fácil, bom desempenho para certos tipos de consulta, modelo de dados mais natural para aplicações orientadas a objetos/JavaScript.
- **Desvantagens:** Menor suporte a transações complexas (comparado a SQL), consistência eventual (em algumas configurações), consultas JOIN podem ser mais complexas ou ineficientes.

Para a secretarIA, a flexibilidade do modelo de documentos foi considerada vantajosa para armazenar dados como eventos de calendário e informações de chat.

2.5.3 Mongoose ODM

Mongoose é uma biblioteca ODM (Object-Document Mapper) para Node.js que fornece uma camada de abstração sobre o MongoDB. Ele permite definir esquemas para os modelos de dados, realizar validações, criar hooks e simplificar as interações com o banco de dados no código da aplicação.

2.6 Segurança em Aplicações Web

A segurança é um aspecto crítico, especialmente em uma aplicação que lida com dados sensíveis como senhas e agendas pessoais.

2.6.1 Autenticação e Autorização (OAuth, JWT, NextAuth.js)

- **Autenticação:** Processo de verificar a identidade de um usuário (quem ele é).
- **Autorização:** Processo de verificar se um usuário autenticado tem permissão para acessar um recurso específico (o que ele pode fazer).

Tecnologias comuns incluem:

- **OAuth:** Um padrão aberto para delegação de acesso, frequentemente usado para login social (ex: "Login com Google").
- **JSON Web Tokens (JWT):** Um padrão para criar tokens de acesso que contêm claims (informações) sobre o usuário, assinados digitalmente para garantir a autenticidade.
- **NextAuth.js:** Uma biblioteca para Next.js que simplifica a implementação de diferentes estratégias de autenticação (Credentials, OAuth, Email, etc.) e gerenciamento de sessão.

2.6.2 Criptografia (Hashing, Simétrica - AES)

- **Hashing:** Processo unidirecional que transforma dados em uma string de tamanho fixo (hash). Usado para armazenar senhas de forma segura (armazena-se o hash, não a senha original). Algoritmos comuns: bcrypt, Argon2.
- **Criptografia Simétrica:** Usa a mesma chave para criptografar e descriptografar dados. Rápida, mas requer o gerenciamento seguro da chave. Algoritmo comum: AES (Advanced Encryption Standard).
- **Criptografia Assimétrica:** Usa um par de chaves (pública e privada). Mais lenta, mas facilita a troca segura de chaves. Usada em HTTPS (TLS/SSL).

Na secretarIA, hashing (bcrypt) foi usado para senhas de login e criptografia simétrica (AES) para as senhas armazenadas no gerenciador.

2.6.3 Principais Vulnerabilidades (OWASP Top 10) e Mitigações

O OWASP Top 10 lista as vulnerabilidades de segurança mais críticas em aplicações web. Algumas relevantes para a secretarIA e suas mitigações incluem:

- **A01: Broken Access Control:** Garantir que usuários só acessem seus próprios dados (verificação de `userId` em todas as operações).
- **A02: Cryptographic Failures:** Usar algoritmos fortes (bcrypt, AES-256), gerenciar chaves de forma segura, usar HTTPS.
- **A03: Injection:** Usar ORM/ODM (Mongoose), validar e sanitizar todas as entradas do usuário.
- **A05: Security Misconfiguration:** Manter dependências atualizadas, configurar headers de segurança HTTP, não expor informações sensíveis em erros.
- **A07: Identification and Authentication Failures:** Usar autenticação multifator (se possível), proteger contra ataques de força bruta, usar gerenciamento de sessão seguro (NextAuth.js).
- **A08: Software and Data Integrity Failures:** Validar dados de fontes externas (APIs), proteger contra XSS para evitar manipulação do lado do cliente.

2.7 Privacidade e Ética em IA

A integração de IA, especialmente LLMs, levanta questões importantes de privacidade e ética.

2.7.1 Preocupações com Privacidade em IPAs e LLMs

IPAs e LLMs processam grandes volumes de dados do usuário, incluindo conversas que podem conter informações pessoais e sensíveis. Preocupações incluem (Mols et al., 2021; Bolton et al., 2021):

- **Coleta Excessiva de Dados:** Os sistemas podem coletar mais dados do que o necessário.
- **Uso Secundário dos Dados:** Os dados podem ser usados para outros fins (treinamento de modelos, publicidade) sem consentimento claro.
- **Vazamentos de Dados:** Falhas de segurança podem expor conversas e dados pessoais.
- **Vigilância:** O potencial de monitoramento constante através de microfones sempre ativos (em smart speakers).
- **Transferência para Terceiros:** Dados enviados para APIs de LLMs (como OpenAI) saem do controle direto do usuário e da aplicação.

É crucial adotar práticas de minimização de dados, transparência, criptografia e obter consentimento informado.

2.7.2 Vieses em Modelos de IA

LLMs treinados em dados da internet podem aprender e perpetuar vieses sociais existentes (de gênero, raça, etc.), levando a respostas injustas ou discriminatórias (Zhao et al., 2023).

2.7.3 Transparência e Explicabilidade (XAI)

LLMs são frequentemente caixas-pretas, tornando difícil entender por que geram uma resposta específica. A falta de explicabilidade (Explainable AI - XAI) pode ser problemática em domínios críticos (Islam et al., 2022).

2.7.4 Regulamentações (LGPD)

A Lei Geral de Proteção de Dados Pessoais (LGPD) no Brasil estabelece regras sobre coleta, armazenamento, tratamento e compartilhamento de dados pessoais, exigindo consentimento, transparência e direitos para os titulares dos dados.

2.8 Experiência do Usuário (UX) em Interfaces Conversacionais

Projetar uma boa experiência em interfaces baseadas em chat ou voz apresenta desafios únicos.

2.8.1 Princípios de Design de CUI/VUI

Princípios importantes incluem (adaptado de Google Design, Amazon Alexa Skills Kit): clareza (o que o sistema pode fazer), eficiência (realizar tarefas rapidamente), tolerância a erros (lidar com entradas ambíguas ou incorretas), feedback (manter o usuário informado), consistência (na linguagem e comportamento) e personalidade (definir um tom apropriado).

2.8.2 Desafios de Usabilidade em Chatbots e IPAs

Desafios comuns incluem (Lopatovska et al., 2020; Budiu, 2018):

- **Descoberta de Funcionalidades:** Usuários podem não saber o que podem pedir.
- **Ambiguidade da Linguagem:** A IA pode interpretar mal a intenção do usuário.
- **Gerenciamento de Erros:** Como o sistema lida com falhas de compreensão ou execução.
- **Manutenção do Contexto:** Lembrar informações de turnos anteriores da conversa.
- **Limitações da IA:** A IA pode não ser capaz de realizar todas as tarefas solicitadas.

2.8.3 Avaliação de Usabilidade (Heurísticas de Nielsen)

As 10 heurísticas de usabilidade de Nielsen (Nielsen, 1994) podem ser adaptadas para avaliar interfaces conversacionais:

1. **Visibilidade do Status do Sistema:** O sistema deve manter o usuário informado (ex: "Estou buscando sua agenda...").
2. **Correspondência entre Sistema e Mundo Real:** Usar linguagem familiar ao usuário.
3. **Controle e Liberdade do Usuário:** Permitir que o usuário corrija erros ou saia de fluxos.
4. **Consistência e Padrões:** Manter a linguagem e o comportamento consistentes.
5. **Prevenção de Erros:** Projetar a conversa para minimizar mal-entendidos.
6. **Reconhecimento em Vez de Lembrança:** Tornar as opções e funcionalidades visíveis ou fáceis de descobrir.
7. **Flexibilidade e Eficiência de Uso:** Permitir atalhos para usuários experientes.
8. **Design Estético e Minimalista:** Evitar informações irrelevantes na conversa.

9. **Ajuda aos Usuários a Reconhecer, Diagnosticar e Recuperar de Erros:** Fornecer mensagens de erro claras e sugestões.
10. **Ajuda e Documentação:** Oferecer ajuda contextual ou exemplos.

Esta fundamentação teórica abrangente fornece o alicerce para compreender as escolhas de design e implementação da secretarIA, bem como para analisar criticamente seus resultados e limitações nos capítulos subsequentes.

Capítulo 3: Metodologia Expandida (Rascunho v3)

Este capítulo detalha a abordagem metodológica empregada no desenvolvimento da aplicação secretarIA, desde a concepção inicial até a análise final. A escolha da metodologia é fundamental para guiar o processo de desenvolvimento, garantir a organização do trabalho e facilitar a adaptação a desafios e novas descobertas ao longo do projeto. Dada a natureza exploratória da integração de tecnologias emergentes como LLMs e LangChain em uma aplicação web full-stack, uma abordagem iterativa e adaptativa foi considerada a mais apropriada.

3.1 Abordagem Metodológica: Desenvolvimento Iterativo e Incremental

O desenvolvimento da secretarIA não seguiu rigidamente um modelo de processo de software formal como Cascata ou Espiral puro. Em vez disso, adotou-se uma **abordagem iterativa e incremental**, inspirada em princípios de metodologias ágeis, embora sem a formalidade completa de frameworks como Scrum ou Kanban, devido à natureza individual do projeto de TCC.

As características principais desta abordagem foram:

- **Iteratividade:** O desenvolvimento ocorreu em ciclos curtos (iterações). Em cada ciclo, um conjunto de funcionalidades ou melhorias era planejado, desenvolvido, testado (informalmente) e integrado à aplicação principal.
- **Incrementalidade:** A aplicação foi construída em partes (incrementos), começando com um núcleo funcional básico (autenticação, estrutura inicial) e adicionando funcionalidades progressivamente (calendário, chat, gerenciador de senhas, integração com IA).
- **Flexibilidade e Adaptação:** A abordagem permitiu ajustar o plano e as prioridades com base no aprendizado obtido em cada iteração, nos desafios técnicos

encontrados (especialmente na integração com a IA) e na evolução da compreensão dos requisitos.
- **Foco na Entrega de Valor:** Cada iteração visava entregar um incremento funcional que agregasse valor à aplicação, mesmo que de forma parcial.
- **Feedback Contínuo (Implícito):** Embora sem um cliente formal, o próprio desenvolvedor atuou como avaliador contínuo, testando as funcionalidades implementadas e identificando pontos de melhoria ou problemas a serem abordados nas iterações seguintes.

Essa abordagem foi escolhida por ser adequada a projetos onde os requisitos podem evoluir, onde há incerteza tecnológica (como a integração com LLMs) e onde é desejável ter um protótipo funcional o mais cedo possível para validação e aprendizado.

3.2 Desenho Metodológico Detalhado: Fases do Projeto

O desenvolvimento pode ser dividido nas seguintes fases principais, que ocorreram de forma parcialmente sobreposta e iterativa:

1. **Fase 1: Pesquisa e Definição (Iteração 0)**
    - **Atividades:** Revisão inicial da literatura sobre IPAs, LLMs, LangChain, Next.js, segurança e UX; definição do escopo inicial do projeto; levantamento dos requisitos funcionais e não funcionais preliminares; escolha da stack tecnológica principal.
    - **Entregas:** Documento de proposta inicial (implícito), lista preliminar de requisitos, definição da arquitetura conceitual.
2. **Fase 2: Configuração do Ambiente e Estrutura Base (Iteração 1)**
    - **Atividades:** Configuração do ambiente de desenvolvimento (Node.js, Git, VS Code); criação do projeto Next.js; configuração do Tailwind CSS; implementação da estrutura básica de pastas; configuração do NextAuth.js para autenticação (inicialmente com Google Provider e/ou Credentials).
    - **Entregas:** Repositório Git inicializado; estrutura do projeto Next.js; funcionalidade básica de login/logout.
3. **Fase 3: Desenvolvimento do Módulo Calendário (Iterações 2-N)**
    - **Atividades:** Escolha e integração de uma biblioteca de calendário React; design da interface do calendário; implementação da conexão com o MongoDB via Mongoose; criação dos modelos de dados (`User`, `Event`); desenvolvimento das API Routes para CRUD de eventos; implementação do modal de adição/edição; testes funcionais informais.

- **Entregas:** Módulo de calendário funcional com visualização e operações CRUD básicas.

4. **Fase 4: Desenvolvimento do Módulo Gerenciador de Senhas (Iterações N+1 - M)**
   - **Atividades:** Design da interface do gerenciador; criação do modelo de dados `Password`; implementação da criptografia AES no backend; desenvolvimento das API Routes para CRUD de senhas; implementação da interface de listagem, adição/edição e visualização segura de senhas; testes funcionais informais.
   - **Entregas:** Módulo gerenciador de senhas funcional com criptografia.
5. **Fase 5: Desenvolvimento do Módulo Chat e Integração IA (Iterações M+1 - P)**
   - **Atividades:** Design da interface de chat; configuração da integração com a API da OpenAI; implementação da lógica de chat no backend usando LangChain (PromptTemplate, ChatOpenAI, LLMChain); busca de contexto da agenda no backend; tratamento de histórico de conversa (se implementado); conexão do frontend com a API de chat; testes da interação conversacional.
   - **Entregas:** Módulo de chat funcional com capacidade de consultar e adicionar eventos via IA.
6. **Fase 6: Refinamento, Testes e Análise (Iterações P+1 - Q)**
   - **Atividades:** Refatoração do código; melhorias na interface e usabilidade com base em autoavaliação; implementação de tratamento de erros mais robusto; revisão das medidas de segurança; testes mais abrangentes das funcionalidades integradas; análise crítica do funcionamento, limitações e desempenho; início da redação dos capítulos de Desenvolvimento e Resultados do TCC.
   - **Entregas:** Protótipo da secretarIA mais estável e polido; rascunhos iniciais dos capítulos de análise.
7. **Fase 7: Redação Final e Documentação (Iteração Final)**
   - **Atividades:** Redação completa de todos os capítulos do TCC, incluindo Introdução, Fundamentação Teórica (expandida), Metodologia, Desenvolvimento, Resultados e Discussão, e Conclusão; formatação do documento; criação de diagramas e captura de telas; revisão final.
   - **Entregas:** Documento final do TCC.

É importante notar que houve ciclos de feedback dentro e entre essas fases. Por exemplo, desafios encontrados na Fase 5 poderiam levar a ajustes na arquitetura definida na Fase 1 ou na implementação de módulos anteriores.

3.3 Ferramentas e Tecnologias: Justificativas Detalhadas

A escolha das ferramentas e tecnologias foi baseada em critérios como popularidade, documentação, ecossistema, adequação aos requisitos do projeto e familiaridade prévia (ou interesse em aprender) do desenvolvedor. As justificativas detalhadas para as principais escolhas foram:

- **Linguagem Principal (JavaScript/TypeScript):** Permite o desenvolvimento full-stack com uma única linguagem, aproveitando o vasto ecossistema NPM. (Nota: Confirmar se TypeScript foi efetivamente utilizado ou apenas JavaScript).
- **Framework Frontend/Backend (Next.js):** Escolhido por sua abordagem integrada full-stack, otimizações de desempenho, renderização híbrida, API Routes e excelente ecossistema React. Simplifica a configuração e o desenvolvimento de aplicações web modernas.
- **Biblioteca UI (React):** Padrão de mercado para construção de interfaces de usuário componentizadas, com vasta comunidade e recursos.
- **Estilização (Tailwind CSS):** Abordagem utility-first que acelera o desenvolvimento de interfaces customizadas e mantém a consistência visual sem a necessidade de escrever muito CSS personalizado.
- **Banco de Dados (MongoDB Atlas):** Banco de dados NoSQL orientado a documentos, escolhido pela flexibilidade de esquema (útil para dados como eventos e chat) e pela facilidade de integração com o ambiente JavaScript/Node.js. A versão Atlas (cloud) simplifica a configuração e o gerenciamento.
- **ODM (Mongoose):** Facilita a interação com o MongoDB no Node.js, fornecendo modelagem de dados, validações e abstração de consultas.
- **Autenticação (NextAuth.js):** Solução completa e segura para autenticação em Next.js, simplificando a implementação de login com credenciais e provedores sociais (Google).
- **LLM (OpenAI GPT-3.5-Turbo/GPT-4):** Modelos de linguagem de ponta com excelente capacidade de compreensão e geração de linguagem natural, acessíveis via API. (Nota: Especificar qual modelo foi predominantemente usado).
- **Framework IA (LangChain):** Simplifica a orquestração da interação com o LLM, gerenciamento de prompts, e potencial integração com outras ferramentas ou memória conversacional.
- **Controle de Versão (Git/GitHub):** Essencial para gerenciamento do código-fonte, colaboração (mesmo individual, para histórico) e backup.
- **Ambiente de Desenvolvimento:** Visual Studio Code (editor de código popular com excelente suporte a JavaScript/TypeScript e extensões), Node.js/NPM (ambiente de execução e gerenciador de pacotes).

- **Ferramenta de Design (Figma - Opcional):** (Nota: Confirmar se alguma ferramenta de design foi usada para prototipagem da UI).

3.4 Coleta e Análise de Dados (Avaliação do Protótipo)

A avaliação do protótipo final da secretarIA envolveu principalmente métodos qualitativos, focados na funcionalidade, usabilidade e na qualidade da interação com a IA:

- **Análise Funcional:** Verificação sistemática se todos os requisitos funcionais definidos (RF01 a RF15) foram implementados corretamente. Isso foi feito através de testes manuais dos fluxos da aplicação (login, cadastro, CRUD de eventos, CRUD de senhas, interação no chat).
- **Avaliação Heurística de Usabilidade:** Autoavaliação da interface utilizando as 10 heurísticas de Nielsen (descritas na Seção 2.8.3) como guia. O objetivo foi identificar potenciais problemas de usabilidade na navegação, clareza da interface, feedback ao usuário e tratamento de erros.
- **Análise da Interação Conversacional:** Testes específicos da funcionalidade de chat, enviando diferentes tipos de comandos e perguntas para a IA (consultas simples e complexas sobre a agenda, comandos de adição/edição/exclusão de eventos) para avaliar a precisão da compreensão da intenção, a capacidade de extrair informações relevantes (datas, horários, títulos), a coerência das respostas e a capacidade de lidar com ambiguidades ou solicitações mal formuladas.
- **Análise de Segurança (Revisão):** Revisão das medidas de segurança implementadas (autenticação, criptografia, controle de acesso) à luz das boas práticas e dos requisitos não funcionais (RNF02, RNF06).

Não foram realizados testes formais com usuários finais devido às limitações de um projeto de TCC individual, mas a análise crítica baseada nesses métodos forneceu insights valiosos sobre os pontos fortes e fracos da aplicação, que são discutidos no Capítulo 5.

3.5 Considerações Éticas

Durante o desenvolvimento, foram consideradas as implicações éticas, especialmente relacionadas à privacidade dos dados do usuário. A decisão de criptografar as senhas armazenadas e a preocupação em minimizar os dados da agenda enviados à API externa da OpenAI refletem essa consideração. A transparência sobre o uso de uma IA externa e a necessidade de consentimento (implícito no uso da funcionalidade de chat) também foram pontos de atenção.

Esta metodologia iterativa e adaptativa, combinada com as ferramentas escolhidas e os métodos de avaliação, permitiu o desenvolvimento sistemático da secretarIA, possibilitando a exploração tecnológica e a análise crítica dos resultados alcançados.

Capítulo 4: Desenvolvimento da Aplicação secretarIA (Rascunho v3)

Este capítulo detalha o processo de construção da aplicação secretarIA, desde a análise aprofundada dos requisitos até a implementação concreta das funcionalidades e da arquitetura de software. Serão descritas as tecnologias utilizadas em cada camada, as decisões de design tomadas, a implementação dos módulos principais (autenticação, calendário, gerenciador de senhas e chat com IA) e os desafios encontrados durante o ciclo de desenvolvimento.

4.1 Análise de Requisitos Detalhada

Com base na definição inicial do problema e nos objetivos do trabalho, foi realizada uma análise mais detalhada para refinar os requisitos funcionais e não funcionais que nortearam o desenvolvimento.

4.1.1 Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as ações e funcionalidades que o sistema deve ser capaz de realizar:

- **RF01: Cadastro de Usuário:** O sistema deve permitir que novos usuários se cadastrem fornecendo e-mail e senha, ou utilizando uma conta Google (OAuth).
- **RF02: Login de Usuário:** O sistema deve permitir que usuários cadastrados façam login utilizando e-mail e senha ou conta Google.
- **RF03: Logout de Usuário:** O sistema deve permitir que usuários autenticados encerrem sua sessão.
- **RF04: Visualizar Calendário:** O sistema deve exibir um calendário interativo (visualização mensal, semanal, diária ou anual - Nota: Confirmar visualizações implementadas) mostrando os eventos do usuário autenticado.
- **RF05: Adicionar Evento Manualmente:** O sistema deve permitir que o usuário adicione um novo evento ao calendário através de um formulário, especificando título, descrição (opcional), data e hora de início e fim.
- **RF06: Editar Evento:** O sistema deve permitir que o usuário modifique os detalhes de um evento existente.

- **RF07: Excluir Evento:** O sistema deve permitir que o usuário remova um evento existente.
- **RF08: Listar Credenciais de Senha:** O sistema deve exibir uma lista das credenciais de senha armazenadas pelo usuário (nome do serviço, nome de usuário), ocultando a senha por padrão.
- **RF09: Adicionar Credencial de Senha:** O sistema deve permitir que o usuário adicione uma nova credencial (serviço, usuário, senha) através de um formulário.
- **RF10: Editar Credencial de Senha:** O sistema deve permitir que o usuário modifique os detalhes de uma credencial armazenada.
- **RF11: Excluir Credencial de Senha:** O sistema deve permitir que o usuário remova uma credencial armazenada.
- **RF12: Enviar Mensagem para IA (Chat):** O sistema deve fornecer uma interface de chat onde o usuário possa digitar mensagens (perguntas ou comandos) em linguagem natural.
- **RF13: Receber Resposta da IA (Chat):** O sistema deve exibir as respostas geradas pela IA na interface de chat de forma clara e organizada.
- **RF14: Consultar Agenda via Chat:** A IA deve ser capaz de processar solicitações do usuário para consultar eventos em sua agenda (ex: "O que tenho hoje?", "Estou livre na sexta à tarde?").
- **RF15: Adicionar Evento via Chat:** A IA deve ser capaz de processar comandos do usuário para adicionar novos eventos à agenda (ex: "Marque uma reunião com a equipe amanhã às 14h"), extraindo os detalhes relevantes da mensagem.

4.1.2 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definem as qualidades e restrições do sistema, influenciando a experiência do usuário e a viabilidade técnica:

- **RNF01: Usabilidade:** A interface do usuário deve ser intuitiva, fácil de aprender e usar, minimizando a carga cognitiva. A interação, tanto gráfica quanto conversacional, deve ser eficiente e satisfatória (conforme ISO 9241-11).
- **RNF02: Segurança:** O sistema deve proteger os dados do usuário contra acesso não autorizado, modificação ou divulgação. Isso inclui autenticação robusta, autorização adequada, criptografia de dados sensíveis (senhas, dados da agenda em trânsito e, idealmente, em repouso) e proteção contra vulnerabilidades web comuns (OWASP Top 10).
- **RNF03: Desempenho:** Os tempos de resposta da interface gráfica e da interação com a IA devem ser aceitáveis para o usuário, evitando longas esperas que causem frustração. O carregamento inicial da página e as transições devem ser rápidos.
- **RNF04: Escalabilidade:** Embora seja um protótipo, a arquitetura deve ser projetada com escalabilidade em mente, permitindo que o sistema lide com um

número crescente de usuários e dados no futuro sem degradação significativa do desempenho (considerando a escalabilidade do Next.js/Node.js e do MongoDB Atlas).
- **RNF05: Manutenibilidade:** O código-fonte deve ser bem organizado, modular, legível, comentado e seguir boas práticas de desenvolvimento para facilitar futuras modificações, correções de bugs e adição de novas funcionalidades.
- **RNF06: Privacidade:** O sistema deve respeitar a privacidade do usuário, minimizando a coleta de dados desnecessários e sendo transparente sobre o uso dos dados, especialmente aqueles enviados para a API da OpenAI. Deve haver conformidade com regulamentações como a LGPD.
- **RNF07: Responsividade:** A interface do usuário deve se adaptar a diferentes tamanhos de tela (desktops, tablets, smartphones), proporcionando uma experiência consistente em todos os dispositivos.
- **RNF08: Confiabilidade:** O sistema deve funcionar de forma consistente e estar disponível quando o usuário precisar. O tratamento de erros deve ser robusto para evitar falhas inesperadas.

4.2 Arquitetura do Sistema Detalhada

Conforme introduzido na Fundamentação Teórica, a secretarIA foi implementada utilizando uma arquitetura full-stack baseada no Next.js, seguindo um modelo cliente-servidor e incorporando princípios da arquitetura de três camadas de forma adaptada ao framework.

(Figura X: Diagrama de Arquitetura Detalhado da secretarIA. Fonte: Autor (2025).)
(Nota: Inserir aqui um diagrama de arquitetura mostrando a interação entre Cliente (Browser), Servidor Next.js (React UI, API Routes), LangChain/OpenAI API e MongoDB.)

- **Visão Geral:** O cliente (navegador) interage com a aplicação Next.js. As páginas são renderizadas no servidor (SSR) ou no cliente (CSR) pelo React. Requisições de dados ou ações são enviadas para as API Routes do Next.js, que rodam no ambiente Node.js no servidor. Essas API Routes constituem a camada de lógica de negócios e acesso a dados, interagindo com o banco de dados MongoDB (via Mongoose) e com a API externa da OpenAI (via LangChain).

- **Camada de Apresentação (Frontend - Next.js/React):**

    - **Estrutura de Pastas:** Uma estrutura organizada foi adotada (ex: `/pages` para rotas, `/components` para componentes reutilizáveis de UI, `/styles` para CSS global/Tailwind config, `/lib` ou `/utils` para funções auxiliares, `/`

`hooks` para custom hooks React, `/context` para gerenciamento de estado global, se aplicável).
    - **Gerenciamento de Estado:** O estado global (informações do usuário logado, talvez preferências) foi gerenciado utilizando a Context API do React (Nota: Confirmar se foi Context API, Zustand ou outra) para compartilhamento entre componentes. O estado local dos componentes (inputs de formulário, estado de carregamento de dados específicos) foi gerenciado com os hooks `useState` e `useEffect`.
    - **Componentização:** A UI foi decomposta em componentes React reutilizáveis (ex: `Layout`, `Navbar`, `CalendarView`, `EventModal`, `ChatWindow`, `ChatMessage`, `PasswordList`, `PasswordItem`, `InputField`, `Button`). Isso promoveu a reutilização de código e a consistência visual.
    - **Roteamento:** O roteamento baseado em arquivos do Next.js (`/pages` directory) foi utilizado para definir as rotas da aplicação (`/`, `/login`, `/dashboard`, `/calendar`, `/chat`, `/passwords`). Rotas protegidas foram implementadas verificando o status da sessão do usuário (obtido via NextAuth.js) antes de renderizar o conteúdo, redirecionando para `/login` se não autenticado.
    - **Comunicação com API:** Funções assíncronas utilizando a `fetch` API nativa (ou `axios` / `SWR` - Nota: Confirmar) foram criadas para interagir com os endpoints das API Routes do backend. O tratamento de estados de carregamento (exibindo spinners ou mensagens) e erros (exibindo mensagens de erro ao usuário) foi implementado nas chamadas à API.

- **Camada de Lógica/Serviços (Backend - API Routes/Express):**
    - **Estrutura das API Routes:** Endpoints RESTful foram criados dentro da pasta `/pages/api/` (ex: `/api/auth/[...nextauth].js`, `/api/events/index.js`, `/api/events/[id].js`, `/api/passwords/index.js`, `/api/passwords/[id].js`, `/api/chat/index.js`). Cada arquivo/rota lida com requisições HTTP específicas (GET, POST, PUT, DELETE) para um determinado recurso.
    - **Middleware:** Funções de middleware foram utilizadas (implicitamente pelo NextAuth.js para autenticação/sessão, e potencialmente customizadas) para tarefas como verificar a autenticação do usuário antes de processar requisições em rotas protegidas, validar dados de entrada (payloads de requisição) e talvez logging.
    - **Serviços de Negócio:** A lógica específica da aplicação foi encapsulada em funções ou módulos de serviço separados (ex: `eventService.js`, `passwordService.js`, `chatService.js`) chamados pelos handlers das

API Routes. Isso incluiu validações (ex: formato de data/hora), interações com o banco de dados e a orquestração da chamada à IA.
    - **Integração com LangChain/OpenAI:** O `chatService.js` (ou similar) implementou a lógica de interação com a IA: receber a mensagem e o histórico, buscar eventos relevantes no `eventService`, construir o prompt usando um `PromptTemplate` do LangChain, instanciar o modelo (`ChatOpenAI`), executar a chain ou chamada direta ao modelo, e processar a resposta antes de retorná-la.

- **Camada de Dados (Persistência - MongoDB/Mongoose):**
    - **Modelos de Dados:** Esquemas Mongoose foram definidos para os principais objetos de domínio: `User` (armazenando e-mail, hash da senha, talvez ID do Google), `Event` (armazenando `userId`, título, descrição, data/hora de início, data/hora de fim) e `Password` (armazenando `userId`, nome do serviço, nome de usuário, senha criptografada). A estrutura pode ser visualizada nas Figuras 18, 19, 20 (se aplicável) e 21.
    - **Operações CRUD:** Funções assíncronas utilizando os métodos do Mongoose (`.create()`, `.find()`, `.findById()`, `.findByIdAndUpdate()`, `.findByIdAndD` foram implementadas nos serviços de backend para realizar as operações de Criar, Ler, Atualizar e Deletar nos documentos do MongoDB, sempre garantindo que as operações fossem restritas aos dados do usuário autenticado (`userId`).
    - **Conexão com Banco de Dados:** Uma conexão segura com a instância do MongoDB Atlas foi estabelecida e gerenciada (geralmente em um arquivo de configuração ou utilitário de banco de dados), utilizando variáveis de ambiente para armazenar a string de conexão.

(Figura 21: Exemplo de Estrutura de Dados no MongoDB (Usuários). Fonte: Autor (2025).) (Nota: Inserir print `21 - banco de dados dos users.png`)

(Figura 19: Exemplo de Estrutura de Dados no MongoDB (Eventos). Fonte: Autor (2025).) (Nota: Inserir print `19 - banco de dados de eventos.png`)

(Figura 18: Exemplo de Estrutura de Dados no MongoDB (Senhas). Fonte: Autor (2025).) (Nota: Inserir print `18 - banco da dados do gerenciador de senhas.png`)

(Figura 20: Exemplo de Estrutura de Dados no MongoDB (Chats - se aplicável). Fonte: Autor (2025).) (Nota: Inserir print `20 - banco de dados chats.png`. Verificar se esta coleção foi realmente usada ou se o histórico era gerenciado de outra forma)

4.3 Implementação das Funcionalidades Chave

A seguir, detalhamos a implementação de alguns dos módulos mais importantes, ilustrando com as telas da aplicação:

- **Autenticação (NextAuth.js):**
  - Configuração do handler principal em `/pages/api/auth/[...nextauth].js`.
  - Definição dos Providers: `CredentialsProvider` para login com e-mail/senha (incluindo a lógica de comparação de hash da senha usando `bcrypt` na função `authorize`) e `GoogleProvider` para login social (configurando `clientId` e `clientSecret` via variáveis de ambiente).
  - Configuração das Callbacks (ex: `session`, `jwt`) para gerenciar o conteúdo da sessão e do token JWT, garantindo que o ID do usuário estivesse disponível.
  - Uso dos hooks do NextAuth.js no frontend (`useSession`, `signIn`, `signOut`) para gerenciar o estado da autenticação, exibir informações do usuário e controlar o fluxo de login/logout. As telas de Login (Figura 1) e Cadastro (Figura 2) foram criadas para interagir com essas funções.
  - Proteção de páginas/rotas no frontend e backend verificando o objeto `session`.

(Figura 1: Tela de Login da Aplicação secretarIA. Fonte: Autor (2025).) (Nota: Inserir print `1 - login tela.png`)

(Figura 2: Tela de Cadastro da Aplicação secretarIA. Fonte: Autor (2025).) (Nota: Inserir print `2 - cadastro tela.png`)

- **Módulo Calendário:**
  - Utilização de uma biblioteca React de calendário (ex: `react-big-calendar`, `FullCalendar` - Nota: Confirmar qual foi usada) para renderizar a visualização interativa, como mostrado na visualização mensal (Figura 3), anual (Figura 6) e em formato de agenda (Figura 7).
  - Busca inicial de eventos do usuário autenticado via chamada à API (`GET /api/events`) ao carregar o componente.
  - Implementação de um componente Modal (provavelmente usando uma biblioteca como `react-modal` ou de um kit de UI como Shadcn/ui) para adicionar/editar eventos (Figura 4). O modal continha um formulário com campos para título, data/hora de início/fim, descrição.
  - Submissão do formulário do modal acionava chamadas à API (`POST /api/events` para criar, `PUT /api/events/[id]` para atualizar), que por sua

vez interagiam com o `eventService` no backend para persistir os dados no MongoDB. O resultado da adição manual pode ser visto na Figura 5.
    - Implementação da funcionalidade de edição (Figura 10) e exclusão (Figura 12), refletindo as mudanças no calendário (Figuras 11 e 13).
    - Atualização da visualização do calendário no frontend após operações de CRUD bem-sucedidas.

(Figura 3: Visualização Mensal do Módulo Calendário. Fonte: Autor (2025).) (Nota: Inserir print `3 - calendario tela.png`)

(Figura 4: Modal de Adição/Edição de Evento. Fonte: Autor (2025).) (Nota: Inserir print `4 - adicionando evento manualmente.png`)

(Figura 5: Calendário com Evento Cadastrado Manualmente. Fonte: Autor (2025).) (Nota: Inserir print `5 - evento cadastrado manualmente.png`)

(Figura 6: Visualização Anual do Módulo Calendário. Fonte: Autor (2025).) (Nota: Inserir print `6 - Aba de ano calendario tela.png`)

(Figura 7: Visualização em Lista (Agenda) do Módulo Calendário. Fonte: Autor (2025).) (Nota: Inserir print `7 - Aba Agenda da tela calendario.png`)

(Figura 10: Exemplo de Edição de Evento. Fonte: Autor (2025).) (Nota: Inserir print `10 - editando evento para outro dia.png`)

(Figura 11: Calendário Após Reagendamento do Evento. Fonte: Autor (2025).) (Nota: Inserir print `11 - evento reagendado para o dia 18.png`)

(Figura 12: Exemplo de Exclusão de Evento. Fonte: Autor (2025).) (Nota: Inserir print `12 - delecao do evento ir ao cinema.png`)

(Figura 13: Calendário Após Exclusão do Evento. Fonte: Autor (2025).) (Nota: Inserir print `13 - tela de calendario apos a delecao do chat.png`)

- **Módulo Gerenciador de Senhas:**
    - Criação de uma interface para listar as credenciais salvas (`GET /api/passwords`), exibindo nome do serviço e nome de usuário (Figura 15). A senha era mascarada por padrão.
    - Implementação de um botão "Revelar/Ocultar" que alternava a visibilidade da senha no frontend (sem armazenar a senha descriptografada permanentemente no estado).
    - Criação de um formulário (talvez em um modal) para adicionar/editar credenciais (Figura 16).

    - **Criptografia no Backend:** No backend (`passwordService`), antes de salvar uma nova senha ou atualizar uma existente, ela era criptografada usando uma biblioteca de criptografia simétrica (ex: `crypto-js` ou o módulo `crypto` nativo do Node.js) com AES-256. A chave de criptografia era derivada da senha mestre do usuário (ou uma chave secreta do servidor - Nota: Detalhar o método exato é crucial para a análise de segurança) e um salt único por credencial (Initialization Vector - IV) era gerado e armazenado junto com o texto cifrado. A descriptografia ocorria no backend apenas quando necessário (ex: para edição, mas não para visualização inicial).
    - Implementação da funcionalidade de exclusão (`DELETE /api/passwords/[id]`). A Figura 17 mostra a lista após um cadastro.

(Figura 15: Interface do Módulo Gerenciador de Senhas. Fonte: Autor (2025).) (Nota: Inserir print `15 - tela gerenciador de senhas.png`)

(Figura 16: Modal de Adição/Edição de Credencial. Fonte: Autor (2025).) (Nota: Inserir print `16 - cadastrar senhas.png`)

(Figura 17: Lista de Senhas Após Cadastro. Fonte: Autor (2025).) (Nota: Inserir print `17 - login e senha cadastrados com sucesso.png`)

- **Módulo Chat e Integração IA:**
    - **Frontend:** Componente `ChatWindow` mantendo o estado do histórico da conversa (`messages`) e o input atual do usuário. Ao enviar uma mensagem (Figura 8), ela era adicionada ao estado `messages` com status "enviando" e uma chamada era feita para `POST /api/chat` com a mensagem atual e, potencialmente, o histórico recente.
    - **Backend (`/api/chat`):**
        1. Recebia a mensagem do usuário e o histórico.
        2. Obtinha o `userId` da sessão.
        3. Chamava o `eventService` para buscar eventos relevantes da agenda do usuário (ex: eventos do dia, da semana seguinte).
        4. Instanciava o `PromptTemplate` do LangChain com as instruções ("Você é a secretarIA..."), o contexto da agenda recuperado e a pergunta/histórico do usuário.
        5. Instanciava o `ChatOpenAI` com a API key (de variável de ambiente).
        6. Executava a chamada ao LLM (ex: `model.invoke(prompt)` ou através de uma `LLMChain`).
        7. Recebia a resposta do LLM.
        8. (Opcional) Pós-processava a resposta (limpeza, formatação).

9. (Potencial) Se a resposta indicasse uma ação (ex: adicionar evento), poderia chamar o `eventService` para persistir a mudança antes de retornar. A Figura 9 ilustra o calendário após um evento ser marcado via chat.
10. Retornava a resposta final para o frontend.

- **Frontend (Continuação):** Ao receber a resposta da API, atualizava o estado `messages`, substituindo a mensagem de "enviando" pela resposta da IA.

(Figura 8: Exemplo de Interação de Adição de Evento via Chat. Fonte: Autor (2025).) (Nota: Inserir print `8 - marcar evento pelo chat com ia.png`)

(Figura 9: Calendário Atualizado Após Interação via Chat. Fonte: Autor (2025).) (Nota: Inserir print `9 - evento marcado depois da interação com o chat.png`)

- **(Opcional) Notificações por E-mail:**
  - (Nota: Se a funcionalidade de notificação por e-mail foi implementada, descrever aqui como foi feita, por exemplo, usando Nodemailer ou um serviço de terceiros como SendGrid, e inserir a Figura 14)

(Figura 14: Exemplo de E-mail de Notificação Recebido (se aplicável). Fonte: Autor (2025).) (Nota: Inserir print `14 - email recebido apos marcar evento.png`. Confirmar se esta funcionalidade foi implementada e descrita)

4.4 Implementação de Segurança e Privacidade

Diversas medidas foram implementadas para atender aos requisitos de segurança (RNF02) e privacidade (RNF06):

- **HTTPS:** Configuração do ambiente de desenvolvimento e (assumindo) deploy para usar HTTPS, garantindo a criptografia dos dados em trânsito.
- **Autenticação Segura:** Utilização do NextAuth.js, que implementa padrões seguros para gerenciamento de sessão (JWTs com assinatura, cookies HttpOnly, SameSite).
- **Hashing de Senhas (Login):** Uso de `bcrypt` para gerar o hash da senha do usuário no momento do registro e para comparar o hash durante o login (via `CredentialsProvider`).
- **Criptografia de Senhas (Gerenciador):** Implementação de criptografia simétrica (AES-256) para as senhas armazenadas no módulo gerenciador, com derivação de chave e uso de IVs, conforme descrito em 4.3.
- **Controle de Acesso:** Todas as operações de CRUD no backend (API Routes) foram implementadas para verificar o `userId` da sessão e garantir que um usuário só pudesse acessar ou modificar seus próprios dados (eventos e senhas).

- **Proteção contra OWASP Top 10 (Tentativas):**
  - **Injeção:** O uso de Mongoose ajuda a prevenir NoSQL Injection, pois ele sanitiza os inputs. A validação de entrada nas API Routes adiciona outra camada.
  - **Autenticação Quebrada:** Mitigada pelo uso do NextAuth.js.
  - **Exposição de Dados Sensíveis:** Mitigada por HTTPS e criptografia/hashing de senhas.
  - **XSS:** Frameworks como React/Next.js possuem mecanismos embutidos para escapar dados antes de renderizá-los no DOM, ajudando a prevenir XSS. A sanitização de inputs no backend também contribui.
  - **Controle de Acesso Quebrado:** Mitigado pela verificação do `userId` em todas as operações de dados.
  - **Componentes Vulneráveis:** A manutenção regular das dependências (via `npm update` ou `yarn upgrade`) é necessária para mitigar riscos de componentes vulneráveis.
- **Segurança da API Key (OpenAI):** A chave da API da OpenAI foi armazenada de forma segura como variável de ambiente no servidor e utilizada apenas no backend (API Route `/api/chat`), nunca exposta no código do frontend.
- **Privacidade (Minimização de Dados):** Na interação com a IA, buscou-se enviar apenas o contexto relevante da agenda (ex: eventos próximos) em vez da agenda inteira, como uma medida de minimização de dados enviados à OpenAI.

4.5 Desafios Enfrentados e Soluções Adotadas

O desenvolvimento de uma aplicação como a secretarIA, que integra múltiplas tecnologias complexas, inevitavelmente apresenta desafios:

- **Desafio 1: Integração Full-Stack com Next.js:** Entender como conectar eficientemente o frontend React com as API Routes do backend, gerenciar o estado da aplicação e lidar com a renderização híbrida exigiu um aprendizado inicial do framework.
  - **Solução:** Estudo da documentação oficial do Next.js, tutoriais e exemplos práticos. Adoção de padrões como o uso de hooks para chamadas à API e gerenciamento de estado local/global.
- **Desafio 2: Gerenciamento de Estado Complexo:** Lidar com o estado da autenticação, os dados do calendário, as senhas e o histórico do chat de forma consistente e eficiente.
  - **Solução:** Utilização do NextAuth.js para o estado da sessão e Context API (ou similar) para estados globais compartilhados. Uso de `useState` e `useEffect` para estados locais dos componentes. Estratégias de atualização

do estado após operações CRUD (ex: re-fetching de dados ou atualização otimista da UI).

- **Desafio 3: Criptografia Segura de Senhas:** Implementar a criptografia das senhas no gerenciador de forma segura, incluindo o gerenciamento adequado das chaves e IVs.
    - **Solução:** Pesquisa sobre boas práticas de criptografia simétrica (AES-256), uso de bibliotecas estabelecidas (`crypto-js` ou módulo nativo `crypto`), geração de IVs únicos por senha e armazenamento seguro da chave de criptografia (derivada da senha mestre ou como variável de ambiente - Nota: Clarificar método exato).
- **Desafio 4: Integração com LangChain e OpenAI API:** Formular prompts eficazes, lidar com as respostas do LLM, extrair informações relevantes (datas, horas), gerenciar o contexto da conversa e otimizar os custos da API.
    - **Solução:** Experimentação iterativa com `PromptTemplate` do LangChain, refinando as instruções e o contexto fornecido. Implementação de lógica no backend para buscar dados da agenda e injetá-los no prompt. Uso de `LLMChain` para simplificar a chamada. Monitoramento do uso da API e otimização dos prompts para reduzir o número de tokens.
- **Desafio 5: Compreensão de Linguagem Natural pela IA:** Lidar com a variabilidade e ambiguidade das solicitações do usuário no chat, garantindo que a IA interpretasse corretamente as intenções e extraísse os parâmetros corretos para as ações na agenda.
    - **Solução:** Engenharia de prompt cuidadosa, fornecendo exemplos e instruções claras ao LLM. Implementação de validações no backend para os dados extraídos pela IA antes de persistir no banco de dados. Aceitação de que a IA pode ter limitações e fornecimento de feedback claro ao usuário em caso de falha na compreensão.
- **Desafio 6: Manter a Qualidade e Manutenibilidade do Código:** Garantir que o código permanecesse organizado, legível e fácil de modificar à medida que a aplicação crescia.
    - **Solução:** Adoção de componentização no React, separação de responsabilidades entre frontend e backend (API Routes), encapsulamento da lógica de negócio em serviços, uso de nomes de variáveis e funções significativas, e adição de comentários em partes complexas do código.

A superação desses desafios foi um processo contínuo de pesquisa, experimentação, depuração e refatoração, fundamental para alcançar o estado atual da aplicação.

Capítulo 5: Resultados e Discussão (Rascunho v3)

Este capítulo apresenta os resultados obtidos com o desenvolvimento da aplicação secretarIA e realiza uma discussão crítica sobre suas funcionalidades, limitações e implicações, conectando os achados práticos com a fundamentação teórica explorada no Capítulo 2. A análise abrange a funcionalidade geral, a interação conversacional com a IA, aspectos de usabilidade, segurança e privacidade.

5.1 Apresentação Detalhada da Aplicação Final

A aplicação secretarIA, ao final do ciclo de desenvolvimento descrito no Capítulo 4, consiste em uma plataforma web acessível via navegador, que integra três módulos principais: Autenticação, Calendário e Gerenciador de Senhas, com a adição de uma interface de Chat para interação com a Inteligência Artificial focada no gerenciamento da agenda.

- **Interface Geral:** A aplicação apresenta um layout consistente, utilizando Tailwind CSS para estilização, com uma barra de navegação superior permitindo acesso aos diferentes módulos após o login (Figuras 1 e 2). A responsividade foi considerada para adaptação a diferentes tamanhos de tela (RNF07).
- **Módulo Calendário:** Oferece visualizações interativas (Figuras 3, 6, 7) e funcionalidades completas de CRUD para eventos (Figuras 4, 5, 10, 11, 12, 13), permitindo ao usuário gerenciar sua agenda de forma gráfica.
- **Módulo Gerenciador de Senhas:** Permite armazenar e visualizar credenciais de forma segura (Figuras 15, 16, 17), com criptografia implementada no backend (RNF02).
- **Módulo Chat:** Disponibiliza uma interface de conversação (Figura 8) onde o usuário pode interagir em linguagem natural com a IA (GPT-3.5-Turbo/GPT-4 via LangChain) para consultar e manipular eventos da agenda (RF12 a RF15), com o calendário refletindo as ações realizadas via chat (Figura 9).

A arquitetura implementada (discutida em 4.2) utiliza Next.js para o frontend e backend (API Routes), React para a UI, MongoDB como banco de dados e integra a API da OpenAI através do LangChain.

5.2 Análise Funcional Detalhada

A Tabela 1 resume a avaliação do cumprimento dos requisitos funcionais (RF) definidos na Seção 4.1.1.

Tabela 1 – Avaliação dos Requisitos Funcionais

RF	Descrição	Status	Observações
RF01	Cadastro de Usuário	**Cumprido**	Implementado com e-mail/senha (bcrypt) e Google OAuth via NextAuth.js.
RF02	Login de Usuário	**Cumprido**	Implementado com e-mail/senha e Google OAuth via NextAuth.js.
RF03	Logout de Usuário	**Cumprido**	Funcionalidade padrão do NextAuth.js implementada.
RF04	Visualizar Calendário	**Cumprido**	Implementado com biblioteca React, exibindo eventos do usuário. Visualizações Mês/Ano/Agenda disponíveis.
RF05	Adicionar Evento Manualmente	**Cumprido**	Implementado via modal e API.
RF06	Editar Evento	**Cumprido**	Implementado via modal e API.
RF07	Excluir Evento	**Cumprido**	Implementado via confirmação e API.
RF08	Listar Credenciais de Senha	**Cumprido**	Interface implementada, senha mascarada por padrão.
RF09	Adicionar Credencial de Senha	**Cumprido**	Implementado via formulário/modal e API, com criptografia no backend.
RF10	Editar Credencial de Senha	**Cumprido**	Implementado via formulário/modal e API, com descriptografia/re-criptografia no backend.
RF11		**Cumprido**	Implementado via confirmação e API.

RF	Descrição	Status	Observações
	Excluir Credencial de Senha		
RF12	Enviar Mensagem para IA (Chat)	**Cumprido**	Interface de chat implementada, enviando mensagem para API backend.
RF13	Receber Resposta da IA (Chat)	**Cumprido**	Respostas da API de chat são exibidas na interface.
RF14	Consultar Agenda via Chat	**Parcialmente Cumprido**	A IA consegue responder a consultas básicas (ex: "eventos de hoje"), mas pode falhar em consultas complexas ou ambíguas.
RF15	Adicionar Evento via Chat	**Parcialmente Cumprido**	A IA consegue extrair detalhes e adicionar eventos para solicitações claras, mas pode errar em datas/horas complexas ou informações incompletas.

Fonte: Autor (2025).

Como observado na Tabela 1, as funcionalidades básicas de autenticação, calendário e gerenciador de senhas foram implementadas com sucesso. Os desafios maiores residem na interação com a IA (RF14 e RF15), que, embora funcional para casos simples, apresenta limitações na robustez da compreensão de linguagem natural, um ponto que será discutido na próxima seção.

5.3 Análise da Interação Conversacional (IA) Aprofundada

A integração da IA conversacional via LangChain e OpenAI API representa o núcleo inovador da secretarIA, mas também a área com maiores desafios e nuances.

- **Capacidades Observadas:**
    - **Consultas Simples:** A IA demonstrou capacidade de responder a perguntas diretas sobre a agenda, como "Quais são meus compromissos de hoje?" ou

"Tenho algo marcado para sexta-feira?". Isso foi possível pela recuperação dos eventos relevantes do MongoDB e sua injeção no prompt do LLM.
    - **Adição de Eventos Claros:** Para comandos bem definidos como "Marcar reunião com equipe às 15h amanhã", a IA conseguiu, na maioria dos testes, extrair a intenção (adicionar evento), o título ("reunião com equipe"), a data ("amanhã") e a hora ("15h"), e acionar a criação do evento no backend.
    - **Manutenção de Contexto Básico:** Com o uso de módulos de memória do LangChain (ou passagem explícita do histórico no prompt), a IA mostrou alguma capacidade de lembrar informações de turnos anteriores da conversa para responder a perguntas de acompanhamento.

- **Limitações e Desafios:**

    - **Ambiguidade e Falta de Informação:** A IA apresentou dificuldades com solicitações vagas ou incompletas (ex: "Marcar almoço semana que vem"). A falta de mecanismos robustos de desambiguação ou de solicitação de esclarecimento ao usuário limitou a eficácia nesses casos. Isso ecoa os desafios de usabilidade em IPAs discutidos por Lopatovska et al. (2020).
    - **Extração de Entidades Complexas:** Datas e horas relativas complexas (ex: "terceira quinta-feira do mês que vem", "1 hora antes do evento X") ou durações não explícitas foram frequentemente mal interpretadas pelo LLM, mesmo com engenharia de prompt.
    - **Alucinações e Respostas Inesperadas:** Em alguns casos, o LLM gerou respostas que pareciam plausíveis mas eram factualmente incorretas sobre a agenda, ou tentou realizar ações não suportadas pela aplicação (alucinações, conforme Zhao et al., 2023). Isso exigiu tratamento de erro no backend para evitar ações indesejadas.
    - **Latência:** O tempo de resposta da IA, envolvendo a chamada à API da OpenAI e o processamento no backend, foi perceptivelmente mais longo do que as interações puramente gráficas, impactando o RNF03 (Desempenho).
    - **Custo da API:** O uso contínuo da API da OpenAI gera custos, o que representa uma limitação para a escalabilidade ou uso gratuito da aplicação em larga escala.

- **Engenharia de Prompt e LangChain:** A experimentação com diferentes templates de prompt no LangChain foi crucial. Foi necessário incluir instruções claras sobre o papel da IA ("Você é a secretarIA..."), o formato esperado das respostas, e fornecer o contexto da agenda de forma estruturada. O uso de `LLMChain` simplificou a orquestração, mas a complexidade da tarefa de gerenciamento de agenda (que envolve estado, tempo e ações concretas) mostrou que apenas um LLM genérico com prompting básico tem suas limitações. Abordagens mais avançadas com

Agentes LangChain (que podem usar ferramentas específicas para buscar datas ou confirmar informações) poderiam melhorar a robustez, mas aumentariam a complexidade da implementação.

Em suma, a interação via IA funciona como um protótipo promissor, demonstrando o potencial dos LLMs para interfaces mais naturais, mas também evidenciando a necessidade de técnicas mais sofisticadas de NLU, gerenciamento de diálogo e integração com ferramentas para lidar com a complexidade do mundo real e as nuances da linguagem humana, corroborando as discussões teóricas sobre as limitações atuais dos LLMs (Seção 2.2.3).

5.4 Avaliação Heurística de Usabilidade Detalhada

Avaliando a interface da secretarIA com base nas heurísticas de Nielsen (Seção 2.8.3):

1. **Visibilidade do Status do Sistema:** Geralmente bom. Indicadores de carregamento foram usados em chamadas de API. O chat mostrava o status da mensagem (enviando/recebido). Poderia melhorar com feedback mais explícito durante operações longas da IA.
2. **Correspondência Sistema-Mundo Real:** Bom. A linguagem usada na UI é direta. O calendário e o gerenciador de senhas seguem convenções conhecidas. A linguagem da IA busca ser natural.
3. **Controle e Liberdade do Usuário:** Razoável. É fácil navegar entre módulos. Operações CRUD possuem confirmação (exclusão). No chat, não há um mecanismo fácil para cancelar uma ação em andamento ou corrigir um comando mal interpretado pela IA antes que ela aja.
4. **Consistência e Padrões:** Bom. O design visual (Tailwind) e a estrutura de navegação são consistentes. A interação nos formulários segue padrões web.
5. **Prevenção de Erros:** Razoável. Validações básicas existem nos formulários. No chat, a prevenção de erros de interpretação pela IA é o maior desafio.
6. **Reconhecimento em Vez de Lembrança:** Bom para UI gráfica (menus claros, calendário visual). Ruim para o chat, pois o usuário precisa lembrar (ou descobrir) o que pode pedir à IA (problema clássico de descoberta em CUI).
7. **Flexibilidade e Eficiência de Uso:** Razoável. A UI gráfica é eficiente para tarefas visuais (ver a semana). O chat poderia ser mais eficiente para adições rápidas, mas as limitações da IA podem negar essa vantagem.
8. **Design Estético e Minimalista:** Bom. A interface é limpa e focada nas tarefas, sem excesso de informação, graças ao Tailwind CSS.
9. **Ajuda na Recuperação de Erros:** Razoável. Erros de formulário são indicados. Erros da IA são geralmente reportados com mensagens genéricas ("Não entendi", "Ocorreu um erro"), sem muita orientação sobre como corrigir.

10. **Ajuda e Documentação:** Fraco. Não há um sistema de ajuda integrado ou documentação para o usuário final (comum em protótipos de TCC).

Conclui-se que a usabilidade da interface gráfica é geralmente boa, seguindo padrões web estabelecidos. A interface conversacional, no entanto, herda os desafios inerentes às CUIs baseadas em LLMs, principalmente relacionados à descoberta de funcionalidades, prevenção/recuperação de erros de interpretação e necessidade de o usuário se adaptar às capacidades (e limitações) da IA.

5.5 Discussão Aprofundada sobre Segurança e Privacidade

A segurança e a privacidade foram consideradas desde o início (RNF02, RNF06), mas a análise revela pontos fortes e áreas de melhoria:

- **Pontos Fortes:**
    - Uso de HTTPS para comunicação.
    - Autenticação robusta via NextAuth.js (hashing de senha, OAuth seguro).
    - Controle de acesso garantindo que usuários só vejam seus dados.
    - Criptografia AES-256 para senhas armazenadas no gerenciador.
    - Armazenamento seguro da API Key da OpenAI no backend.
- **Áreas de Melhoria e Preocupações:**
    - **Gerenciamento da Chave de Criptografia AES:** O método exato de derivação/armazenamento da chave para criptografar as senhas no gerenciador é crítico. Se a chave for fixa no código ou facilmente derivável, a segurança é comprometida. Idealmente, deveria ser derivada da senha mestre do usuário (exigindo que ele a digite para descriptografar) ou usar mecanismos mais avançados de gerenciamento de chaves. (Nota: Detalhar a implementação exata aqui é crucial).
    - **Privacidade dos Dados Enviados à OpenAI:** Embora o contexto enviado tenha sido minimizado, o fato é que dados da agenda do usuário (títulos, datas, horários) são enviados para um serviço de terceiros. Isso representa um risco de privacidade inerente ao uso de APIs de LLM externas, conforme discutido por Mols et al. (2021). Uma alternativa seria usar modelos open-source hospedados localmente (com desafios de custo e desempenho) ou implementar técnicas mais avançadas de anonimização/pseudonimização antes de enviar os dados.

- **Segurança do Banco de Dados:** A segurança da instância do MongoDB Atlas depende das configurações da plataforma e do gerenciamento adequado das credenciais de acesso (armazenadas como variáveis de ambiente).
- **Vulnerabilidades de Dependências:** A segurança geral depende da manutenção e atualização regular das bibliotecas de terceiros utilizadas (Next.js, NextAuth.js, Mongoose, LangChain, etc.).

Em resumo, foram implementadas medidas de segurança padrão importantes. No entanto, a criptografia do gerenciador de senhas requer uma análise mais profunda da gestão da chave, e a privacidade na interação com a IA externa permanece uma preocupação inerente ao modelo de arquitetura escolhido.

Esta análise dos resultados e a discussão das capacidades e limitações da secretarIA fornecem a base para as conclusões e recomendações de trabalhos futuros apresentadas no próximo capítulo.

Capítulo 6: Conclusão (Rascunho v3)

Este capítulo finaliza o trabalho, sintetizando os principais achados, respondendo aos objetivos propostos, destacando as contribuições, reconhecendo as limitações e sugerindo caminhos para pesquisas e desenvolvimentos futuros baseados na experiência adquirida com o projeto secretarIA.

6.1 Síntese Abrangente do Trabalho

Este Trabalho de Conclusão de Curso abordou o projeto, desenvolvimento e análise da "secretarIA", uma aplicação web concebida como um assistente pessoal inteligente com foco no gerenciamento integrado de agendas e senhas. Motivada pela crescente demanda por ferramentas de produtividade mais eficientes e pela rápida evolução da inteligência artificial conversacional, a secretarIA buscou explorar a viabilidade e os desafios de combinar uma interface gráfica tradicional (para calendário e gerenciador de senhas) com uma interface conversacional (chat) alimentada por um Modelo de Linguagem Grande (LLM) para interação em linguagem natural.

O desenvolvimento seguiu uma abordagem iterativa e incremental, utilizando uma stack tecnológica moderna baseada em JavaScript/TypeScript, com Next.js (React/Node.js) para a estrutura full-stack, MongoDB como banco de dados NoSQL, e integração com a API da OpenAI (GPT-3.5-Turbo/GPT-4) através do framework LangChain para as funcionalidades de IA. Foram implementados módulos para autenticação segura de usuários, gerenciamento visual e CRUD de eventos de calendário, armazenamento

seguro e CRUD de credenciais de senha (com criptografia), e uma interface de chat permitindo consultar e adicionar eventos à agenda via comandos em linguagem natural.

A análise dos resultados revelou que as funcionalidades básicas da aplicação foram implementadas com sucesso, oferecendo uma plataforma unificada para as tarefas propostas. A integração com a IA demonstrou potencial para simplificar certas interações, como a adição rápida de eventos, mas também evidenciou limitações significativas na compreensão de linguagem natural complexa ou ambígua, na extração precisa de entidades e na robustez geral da conversação. A avaliação de usabilidade indicou uma boa experiência na interface gráfica, mas apontou desafios comuns às interfaces conversacionais, como descoberta de funcionalidades e tratamento de erros de interpretação. As considerações de segurança e privacidade foram abordadas com medidas padrão, embora a gestão da chave de criptografia e a privacidade dos dados enviados à IA externa permaneçam pontos de atenção.

6.2 Resposta aos Objetivos Específicos

Avaliando o alcance dos objetivos específicos definidos na Seção 1.3:

1. **Pesquisar e Analisar: Cumprido.** Foi realizada uma extensa revisão da literatura (Capítulo 2) sobre IPAs, LLMs, LangChain, Next.js, MongoDB, segurança, privacidade e UX, fornecendo a base teórica para o projeto.
2. **Projetar a Arquitetura: Cumprido.** Uma arquitetura full-stack baseada em Next.js, MongoDB e OpenAI API foi definida e documentada (Seção 4.2).
3. **Desenvolver a Aplicação: Cumprido.** As funcionalidades principais de autenticação, calendário e gerenciador de senhas foram implementadas com sucesso (Seção 4.3).
4. **Integrar a Inteligência Artificial: Parcialmente Cumprido.** A lógica de IA conversacional foi integrada usando LangChain e OpenAI API, permitindo consultas e adições básicas à agenda via chat. No entanto, a robustez e a capacidade de lidar com casos complexos foram limitadas (Seções 4.3 e 5.3).
5. **Avaliar e Analisar: Cumprido.** Foi realizada uma análise crítica da aplicação, discutindo funcionalidades, limitações (especialmente da IA), desafios e aspectos de usabilidade e segurança à luz da teoria (Capítulo 5).

Conclui-se que os objetivos principais relacionados à pesquisa, projeto e desenvolvimento da estrutura básica da aplicação foram atingidos. O objetivo de integrar a IA foi parcialmente atingido, demonstrando a funcionalidade básica, mas revelando a complexidade de se obter uma interação conversacional verdadeiramente robusta e confiável para tarefas de gerenciamento de agenda com as abordagens implementadas.

6.3 Principais Contribuições

Este trabalho oferece as seguintes contribuições:

- **Estudo de Caso Prático:** Fornece um exemplo detalhado do desenvolvimento de uma aplicação web full-stack moderna (Next.js, React, Node.js, MongoDB) que integra um LLM (via LangChain e OpenAI API) para uma tarefa específica de produtividade pessoal (gerenciamento de agenda e senhas).
- **Análise da Integração LLM-Aplicação:** Oferece insights sobre os desafios e limitações práticas da integração de LLMs genéricos em aplicações que exigem compreensão contextual e execução de ações concretas, destacando a importância da engenharia de prompt, do tratamento de erros e das limitações inerentes aos modelos atuais.
- **Exploração de Arquitetura:** Demonstra uma arquitetura viável para combinar interfaces gráficas e conversacionais dentro de um mesmo framework (Next.js).
- **Discussão sobre Segurança e Privacidade em IPAs com LLMs:** Levanta e discute questões relevantes sobre a segurança (criptografia de dados locais) e a privacidade (dados enviados a APIs externas) no contexto específico da aplicação desenvolvida.
- **Consolidação de Conhecimento:** Sistematiza a aplicação de diversos conceitos e tecnologias abordados ao longo do curso de Ciência da Computação em um projeto integrado.

6.4 Implicações Práticas e Teóricas

- **Práticas:** O protótipo da secretarIA demonstra o potencial de interfaces híbridas (gráfica + conversacional) para ferramentas de produtividade. Desenvolvedores que buscam integrar LLMs em suas aplicações podem se beneficiar dos desafios e soluções discutidos neste trabalho, especialmente em relação à engenharia de prompt e ao tratamento das limitações dos modelos. A necessidade de criptografia robusta para dados sensíveis como senhas é reforçada.
- **Teóricas:** O trabalho corrobora achados da literatura sobre os desafios de usabilidade, segurança e privacidade em IPAs e LLMs. Ele evidencia a lacuna entre as capacidades impressionantes de geração de linguagem dos LLMs e sua habilidade de realizar tarefas complexas, dependentes de contexto e que exigem raciocínio preciso ou interação com o mundo real (como manipular uma agenda de forma confiável). Sugere que abordagens mais sofisticadas (como agentes com ferramentas ou modelos especializados) podem ser necessárias para superar essas limitações.

6.5 Limitações e Desafios Remanescentes

O presente trabalho possui limitações inerentes ao escopo de um TCC e à complexidade das tecnologias envolvidas:

- **Avaliação de Usabilidade:** Não foram realizados testes formais com usuários finais, limitando a validação da usabilidade a uma avaliação heurística pelo próprio desenvolvedor.
- **Robustez da IA:** A capacidade da IA de compreender e agir sobre comandos de agenda é limitada a casos mais simples, sendo suscetível a erros de interpretação.
- **Funcionalidades Avançadas:** Recursos como eventos recorrentes, compartilhamento de calendário, notificações mais inteligentes ou integração com outras plataformas não foram implementados.
- **Segurança da Criptografia:** A segurança real do gerenciador de senhas depende criticamente da implementação específica do gerenciamento da chave AES, que necessita de maior detalhamento e validação.
- **Escalabilidade e Custo:** A aplicação não foi testada sob carga e o custo da API da OpenAI pode ser proibitivo para uso em larga escala.
- **Tratamento de Erros:** O tratamento de erros, especialmente na interação com a IA, poderia ser mais robusto e informativo para o usuário.

6.6 Recomendações Detalhadas para Trabalhos Futuros

Com base nas limitações e aprendizados, sugerem-se as seguintes direções para trabalhos futuros:

1. **Melhorar a Robustez da IA:**
    - Explorar o uso de **Agentes LangChain** com ferramentas específicas para análise de datas/horas, consulta à agenda e confirmação de informações antes de agir.
    - Investigar técnicas de **Fine-tuning** de modelos menores (open-source) com dados específicos de gerenciamento de agenda para melhorar a precisão e reduzir custos.
    - Implementar mecanismos de **desambiguação** e solicitação de esclarecimento no diálogo quando a intenção do usuário não for clara.
2. **Aprimorar a Usabilidade:**
    - Realizar **testes formais com usuários** para coletar feedback quantitativo e qualitativo sobre a interface gráfica e conversacional.
    - Melhorar a **descoberta de funcionalidades** do chat, talvez com sugestões de comandos ou um tutorial interativo.
    - Aprimorar o **feedback e o tratamento de erros** na interação com a IA.

3. **Expandir Funcionalidades:**
    - Adicionar suporte a **eventos recorrentes**.
    - Implementar **compartilhamento de calendário** entre usuários.
    - Desenvolver um sistema de **notificações** mais inteligente (ex: lembretes antes de eventos, sugestões baseadas na agenda).
    - Integrar com **APIs de terceiros** (ex: Google Calendar, Outlook Calendar).
4. **Reforçar a Segurança e Privacidade:**
    - Implementar um método mais seguro para **gerenciamento da chave de criptografia AES**, como derivação da senha mestre do usuário (com KDFs como PBKDF2 ou Argon2) ou uso de um Hardware Security Module (HSM) em um cenário de produção.
    - Explorar **modelos de IA open-source auto-hospedados** para eliminar a dependência de APIs externas e melhorar a privacidade.
    - Implementar **autenticação de dois fatores (2FA)**.
5. **Otimizar o Desempenho e Escalabilidade:**
    - Realizar **testes de carga** para identificar gargalos.
    - Otimizar as **consultas ao banco de dados**.
    - Implementar **caching** para dados frequentemente acessados.

6.7 Considerações Finais e Visão de Futuro

O desenvolvimento da secretarIA representou uma jornada desafiadora e recompensadora na exploração da interseção entre desenvolvimento web moderno e inteligência artificial conversacional. O projeto demonstrou ser viável construir uma aplicação funcional que integra gerenciamento de agenda, senhas e interação via LLM, oferecendo uma visão do potencial futuro dos assistentes pessoais.

Embora as limitações atuais da tecnologia de IA, especialmente em termos de robustez e compreensão contextual para tarefas complexas, tenham sido evidentes, os avanços contínuos em LLMs, frameworks como LangChain e técnicas de engenharia de prompt sugerem que essas barreiras podem ser gradualmente superadas. A chave para o sucesso futuro de aplicações como a secretarIA reside não apenas na melhoria da tecnologia de IA subjacente, mas também em um design cuidadoso focado na experiência do usuário, na segurança robusta e em considerações éticas rigorosas, especialmente em relação à privacidade dos dados.

Espera-se que este trabalho contribua para a discussão acadêmica e prática sobre o desenvolvimento de assistentes pessoais inteligentes especializados e que as lições aprendidas possam informar projetos futuros nesta área promissora.

(MARCADOR - REFERÊNCIAS BIBLIOGRÁFICAS)

REFERÊNCIAS BIBLIOGRÁFICAS

(Espaço para a lista completa de referências citadas no texto, formatada conforme as normas ABNT ou da instituição. Incluir as referências originais fornecidas pelo usuário, as encontradas na pesquisa adicional e quaisquer outras citadas durante a expansão.)

Exemplos (Formato ABNT - verificar normas específicas):

BOLTON, T.; DHANDA, G.; RAJAKUMAR, A. Privacy concerns in voice assistants: A systematic literature review. Computers & Security, v. 108, p. 102350, 2021.

BÉRUBÉ, C.; FOURNIER, P. A.; LAI, N. Y.; et al. Intelligent personal assistants for health care: systematic review. Journal of Medical Internet Research, v. 23, n. 4, p. e25312, 2021.

BUDIU, R. The User Experience of Voice Assistants. Nielsen Norman Group, 2018. Disponível em: https://www.nngroup.com/articles/voice-assistant-ux/. Acesso em: [Data].

FIELDING, R. T. Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000.

FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2007.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.

GUBARAVA, T.; LOPES, M. Intelligent personal assistants in education: a systematic literature review. In: INTERNATIONAL CONFERENCE ON ADVANCED LEARNING TECHNOLOGIES (ICALT), 20., 2020, Tartu. Proceedings [...]. IEEE, 2020. p. 355-357.

HAN, A.; YANG, H. Past, Present, and Future of Intelligent Personal Assistants: A Systematic Literature Review. International Journal of Human–Computer Interaction, v. 34, n. 12, p. 1121-1141, 2018.

HOY, M. B. Alexa, Siri, Cortana, and More: An Introduction to Voice Assistants. Medical Reference Services Quarterly, v. 37, n. 1, p. 81-88, 2018.

ISLAM, M. U.; et al. Explainable Artificial Intelligence for Cyber Security. IEEE Access, v. 10, p. 131879-131902, 2022.

ISLAM, M. U.; CHAUDHRY, B. M. A Framework to Enhance User Experience of Older Adults With Speech-Based Intelligent Personal Assistants. IEEE Access, v. 10, p. 134336-134351, 2022.

KUNDU, S. Large Language Model (LLM): The Ultimate Guide (2023). V7 Labs Blog, 2023. Disponível em: https://www.v7labs.com/blog/large-language-models-llms. Acesso em: [Data].

LOPATOVSKA, I.; et al. User Experience with Intelligent Personal Assistants: A Comparative Lab Study. Proceedings of the ACM on Human-Computer Interaction, v. 4, n. CSCW1, p. 1-28, 2020.

MOLS, A.; et al. Household intelligent personal assistants in the Netherlands: Exploring privacy concerns around surveillance, security, and platforms. Convergence: The International Journal of Research into New Media Technologies, v. 28, n. 1, p. 198-216, 2021. (Publicado online em 2021, edição impressa 2022).

NIELSEN, J. Heuristic evaluation. In: NIELSEN, J.; MACK, R. L. (Eds.). Usability inspection methods. New York: John Wiley & Sons, 1994.

NOEL, V. Intelligent personal assistants for accessibility: A systematic literature review. Disability and Rehabilitation: Assistive Technology, v. 15, n. 7, p. 737-750, 2020.

PIÑEIRO-MARTÍN, N.; et al. Ethical implications of large language models: A systematic literature review. AI and Ethics, 2023. (Publicação antecipada online).

RILLIG, M. C.; et al. Risks and benefits of large language models for the environment. Environmental Science & Technology, v. 58, n. 1, p. 179-188, 2024.

SILVA, J.; et al. Intelligent Personal Assistants: a systematic literature review. Expert Systems with Applications, v. 147, p. 113193, 2020.

VASWANI, A.; et al. Attention is all you need. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NIPS), 30., 2017, Long Beach. Proceedings [...]. Curran Associates, Inc., 2017. p. 5998-6008.

WEIZENBAUM, J. ELIZA—a computer program for the study of natural language communication between man and machine. Communications of the ACM, v. 9, n. 1, p. 36-45, 1966.

ZHAO, W. X.; et al. A Survey of Large Language Models. arXiv preprint arXiv:2303.18223, 2023.

...(Adicionar outras referências encontradas e citadas)

(MARCADOR - FIM DAS REFERÊNCIAS)

(MARCADOR - APÊNDICES - se houver)

(APÊNDICE A – Exemplo de Código)

(Seção opcional para incluir trechos de código relevantes, configurações, etc.)

(MARCADOR - FIM DOS APÊNDICES)

(MARCADOR - FIM DO DOCUMENTO)